

摘要

新冠疫情之后,非接触式体温测量已经成为公共卫生防控的常规手段,各类公共场所对小型化、低功耗、响应快速的红外测温设备需求量极大。本文围绕一款手持式红外测温仪的电路系统展开设计,以STM32F103C8T6单片机为核心,集成红外测温传感器、OLED显示屏、按键、蜂鸣器以及RGB指示灯等模块,完成了从硬件电路、PCB布板到上位机软件的全套设计与调试。系统采用GY-614V3型非接触红外测温模块,通过串口通讯获取目标温度、人体体温和环境温度三组数据,测温范围覆盖0至60摄氏度,人体测温段精度达到 ± 0.2 摄氏度。软件层面采用FreeRTOS实时操作系统,以2毫秒为基础周期通过计数器取余的方式实现多级任务调度,同时设计了报警阈值可调、低电量图标闪烁、30秒无操作自动休眠等贴近实际使用的功能。整机PCB尺寸仅为38mm $\times$ 28mm,集成度高,具备一定的商业化潜力。经过实测,系统从按键到稳定显示用时不超过5秒,各项功能均达到预期指标,可满足家庭、学校、办公场所等场景下的非接触测温需求。

关键词: 红外测温、STM32、FreeRTOS、OLED显示、低功耗

Abstract

After the COVID-19 pandemic, non-contact body temperature measurement has become a routine means of public health prevention and control. Public places of all kinds have a huge demand for miniaturized, low-power, and fast-responding infrared temperature measurement equipment. This paper focuses on the circuit system design of a handheld infrared thermometer. With the STM32F103C8T6 microcontroller as the core, it integrates an infrared temperature sensor, an OLED display, buttons, a buzzer, and an RGB indicator. The full set of design and debugging works including hardware circuit, PCB layout, and lower-machine software has been completed. The system adopts the GY-614V3 non-contact infrared temperature measurement module, obtaining three sets of data through serial communication: target temperature, human body temperature, and ambient temperature. The temperature measurement range covers 0 to 60 degrees Celsius, with an accuracy of ± 0.2 degrees Celsius in the human body measurement range. On the software side, the FreeRTOS real-time operating system is used, with a 2ms base cycle. Multi-level task scheduling is realized through the counter modulo method. At the same time, practical functions such as adjustable alarm threshold, low-power icon blinking, and automatic sleep after 30 seconds of no operation are designed. The overall PCB size is only 38mm $\times$ 28mm, with high integration and certain commercial potential. Through actual measurement, the system takes no more than 5 seconds from button press to stable display, and all functions have achieved the expected indicators, which can meet the non-contact temperature measurement needs in family, school, office and other scenarios.

Keywords: Infrared Temperature Measurement, STM32, FreeRTOS, OLED Display, Low Power Consumption

第一章 绪论

1.1 课题的研究背景与意义

近些年来,公共卫生事件频频发生,体温作为人体最基本的健康指标之一,其快速、准确的测量已经成为疫情防控、健康监测、医疗诊断中不可或缺的一环。传统的水银温度计虽然精度高、成本低,但是测量时间长、需要接触人体、存在汞污染风险,在大规模人群筛查的场合下显然力不从心。电子体温计虽然解决了汞污染的问题,但是仍然需要接触测量,使用上不太方便,而且也存在交叉感染的隐患。在这种背景下,基于红外辐射原理的非接触式测温技术迅速进入了公众视野。

笔者在做这个课题之前,其实对红外测温的具体原理也只是一知半解,只知道红外测温仪可以隔空测温、响应很快。后来翻阅了一些资料才明白,任何温度高于绝对零度的物体都会向外辐射红外线,辐射的强度与物体温度的四次方成正比,这就是著名的斯蒂芬-玻尔兹曼定律。红外测温仪的工作原理就是通过热电堆传感器或者MEMS红外传感器来捕获被测物体表面发射的红外辐射,经过光电转换之后再通过内部的算法补偿环境温度、发射率等因素,最终得出物体表面的真实温度。整个过程不需要接触被测物体,响应时间通常在零点几秒到几秒之间,啊,这相比于传统的水银温度计来说,效率提升是巨大的。

红外测温仪的应用场景也非常广泛,不仅仅局限于人体测温。在工业领域,它可以用于电力设备的温度巡检、电机轴承的过热预警、冶金行业的熔体温度监控等场合。在家庭场景中,它可以用来测量水温、奶温、烹饪温度等。在医疗场景中,它可以用于发热筛查、术中监测等。可以说,只要是涉及温度测量的场合,红外测温仪都有一席之地。

我做这个毕业论文的初衷,一方面是想把大学四年学到的电子电路、嵌入式开发、单片机原理等知识做一次系统性的整合应用,另一方面也是想做一个真正具有实用价值的小产品,而不是单纯为了应付答辩的"实验台"。所以本课题不只是简单地把传感器和单片机连起来读个温度数显示出来,而是从用户实际使用的角度出发,把电池供电、按键交互、报警功能、低功耗休眠、OLED图形显示这些产品级的特性都纳入了考虑。这种做法虽然增加了开发难度,但是对锻炼综合工程能力非常有帮助,我觉得这才是毕业设计应有的样子吧。

1.1.1 红外测温仪国内外研究现状

放眼全球,红外测温技术的研究和产业化已经走过了相当长的一段时间。国外起步比较早,德国Heimann公司、美国Melexis公司、德州仪器TI、瑞士的洛桑联邦理工学院都在这一领域有深厚的积累。其中,Melexis公司推出的MLX90614系列红外测温传感器是这个领域里的经典产品,它采用了双通道红外热电堆加内置DSP的架构,通过I2C接口对外输出温度数据,精度可以做到 ± 0.5 摄氏度甚至更高,广泛应用于消费电子、汽车电子、工业自动化等场合。Heimann公司则在阵列式红外传感器方面有独到之处,其HTPA系列芯片可以实现 32×32 甚至更高分辨率的热成像,主要应用于高端的红外热像仪产品。国外的成品方面,博朗(Braun)、欧姆龙(OMRON)、福禄克(FLUKE)等品牌的红外测温仪在精度、稳定性、可靠性方面都做得非常出色,但是价格也相对偏高,一般在几百到上千人民币不等。

国内在红外测温技术方面起步稍晚一些,但是这些年发展势头非常迅猛。特别是2020年新冠疫情爆发之后,国内涌现出了一大批红外测温产品的生产厂家,从几十块钱的家用额温枪到几万元的大型门禁测温系统,产品线几乎覆盖了所有应用场景。在传感器层面,深圳的迈来芯、湖南的圣湘生物、上海的烨映微电子等公司都推出了具有自主知识产权的红外测温芯片或者模组。在成品层面,鱼跃医疗、海尔、九安、博朗(部分中国生产)等品牌的红外测温仪在国内市场占有率非常高。国内学术界对红外测温的研究也从未停止过,清华大学、中科院、华中科技大学等高校在红外探测器件、温度补偿算法、热成像技术方面都有不少高水平的研究成果。

具体到本课题选用的GY-614V3模块,它实际上是国内厂家基于Melexis的MLX90614芯片二次封装的小型化模组,在原有I2C接口的基础上增加了串口转换电路,使得用户可以直接通过UART接口读取温度数据,大大降低了使用门槛。这种"芯片+模组"的产业链分工,使得普通的电子爱好者和学生也能很方便地用上工业级的红外测温器件,啊,这真是我们这一代电子工程师的幸运。

不过,现有的红外测温产品也不是没有缺点。一是精度问题,虽然厂家标称的精度都很高,但是实际使用中往往会受到环境温度、被测物体距离、表面发射率等因素的影响,在不同条件下读数会有偏差。二是功能比较单一,大多数产品只能做温度显示,缺乏数据存储、阈值报警、远程通信等增值功能。三是用户体验有提升空间,一些产品的UI设计比较粗糙,按键反馈不够清晰,休眠唤醒逻辑不够智能。这些都是本课题在设计时希望去尝试改进的方向。

1.1.2 红外测温仪的价值与意义

红外测温仪虽然看上去是一个不起眼的小设备,但是它的社会价值和工程意义都不容小觑。

从社会价值的角度看,红外测温仪在公共卫生领域扮演着第一道防线的角色。在火车站、机场、学校、医院、办公楼等人员密集的场所,它能够以非接触的方式快速完成大量人员的体温筛查,在传染病防控中发挥关键作用。疫情期间,我们国家正是依靠遍布全国各地的红外测温设备,才得以在短时间内建立起严密的防控网络。除了应急防疫之外,红外测温仪在日常的健康监测中也越来越普及,很多家庭都备有一两支额温枪,用于给小孩、老人测量体温,既方便又卫生。

从工程意义的角度看,红外测温仪是一个非常典型的小型嵌入式产品,涉及到的技术点非常全面。硬件方面,它包含了电源管理、传感器接口、人机交互、显示驱动、声光报警等多个子模块,这些都是嵌入式开发中最常见也最基础的技术。软件方面,它涉及到串口通信、状态机解析、定时任务调度、低功耗管理、UI刷新策略等知识点,既考验底层驱动能力又考验应用层架构能力。机械结构方面,它要兼顾PCB布局、电池容纳、按键外露、显示屏对齐等多个约束条件,对结构设计能力也是一种锻炼。所以说,做一个完整的红外测温仪项目,对一个嵌入式工程师来说,基本上可以把本科阶段学到的知识全部串起来用一遍,这种综合训练对个人能力的提升是非常显著的。

笔者在这个课题中,把整机PCB做到了38mm×28mm的尺寸,这个尺寸已经接近商业化产品的水平了。在这么小的板上要塞下STM32最小系统、LDO稳压电路、红外传感器接口、OLED接口、按键、蜂鸣器、RGB灯、电池电压采样电路,布局布线的难度相当大。这对锻炼PCB设计能力、提升空间利用率意识非常有帮助。同时,我设计的软件采用了分层式的代码结构,把代码组织为应用层、驱动层和基础库三层,各层之间通过明确的接口进行解耦,这种工程化的设计思想对今后从事嵌入式开发也是大有裨益的。

总而言之,本课题虽然在技术深度上不能与商业化的高端产品相提并论,但是从一个学生作品的角度来说,它涵盖了电子工程师必备的多项核心技能,具备完整的功能闭环和良好的用户体验,有一定的实用价值,也为今后的产品迭代留下了充足的空间。

第二章 系统方案设计

2.1 需求分析

在动手画原理图之前,我花了不少时间去梳理这个课题到底要做什么样子。任务书上虽然给出了六条具体的功能要求,但是要把这些要求落实到具体的器件选择、电路结构、软件逻辑上,还需要做更细致的分析和拆解。下面我从功能需求、性能需求、用户体验需求三个维度来逐条梳理本系统的需求。

功能需求方面,核心需求有六项。第一是非接触式测温,系统要能够通过红外传感器测量并显示被测物体的表面温度,这是整个系统存在的根本目的。第二是合适的测温范围,设定为32至45摄氏度,可扩展至0至60摄氏度,在人体测温范围内精度要达到±0.2摄氏度,这就要求传感器本身要够准,同时单片机的数据处理也不能引入太大的误差。第三是温度报警功能,要通过按键自行设定上下限报警阈值,设定步进为0.1摄氏度,超出范围时立刻通过蜂鸣器和红色LED发出声光报警,这一条决定了人机交互的基本框架。第四是测量响应迅速,从按下测量键到温度值稳定显示在屏幕上的整个过程不超过5秒,这给软件的实时性提出了要求。第五是LCD液晶显示,要清晰显示实时测量温度值、设定的报警阈值、电池电量图标以及超温报警提示信息,这就要求显示屏要有足够的尺寸和分辨率。第六是数据保持与自动关机,测量完成后温度值要保持显示至少10秒,系统在任何操作30秒后要自动进入低功耗休眠模式,这既是为了省电也是为了用户使用方便。

性能需求方面,我归纳了四项关键指标。一是测温精度,人体段要做到±0.2摄氏度,这其实是一个相当严苛的指标,几乎所有商业产品都是按这个标准来标注的。二是响应时间,5秒内完成完整测量流程,这要求传感器读取、数据处理、显示刷新这一整条链路都要够快。三是低功耗,30秒进入休眠后整机功耗要尽可能低,以延长电池续航时间。四是PCB集成度,在保证功能完整的前提下尽可能小型化,以便后续装入手持外壳。

用户体验需求方面,虽然任务书没有明文写,但是我觉得一个好的产品必须考虑用户的实际使用感受。比如说按键的响应必须及时,不能有按了没反应的情况;显示界面要信息齐全又不至于拥挤;低电量时要有提示,免得用户突然发现没电了一脸懵;唤醒后要立刻显示当前状态,而不是先黑屏再慢慢加载。这些细节虽然不会出现在评分表上,但是它们决定了一个产品能不能被持续使用下去。

把上面这些需求汇总起来之后,我才能进行下一步的方案选型。下面就来对几种可能的设计方案做一个对比分析。

2.2 系统设计方案对比

在确定最终方案之前,我考虑过好几种不同的实现路径,这些方案在主控选择、传感器接口、显示方式上各有侧重。下面我把其中比较有代表性的两个备选方案与最终方案做一个对比,说明为什么最后选了现在这一套。

方案一:基于51单片机+模拟红外+段码LCD的传统方案。这个方案使用最经典的STC89C52单片机作为主控,搭配模拟输出型的热电堆传感器(比如MLX90247),通过运放放大之后再用ADC采集,显示部分使用段码式的LCD屏(比如LCD1602)。这个方案的优点是成本极低,所有元器件加起来不到二十元,而且开发难度不大,网上也有大量的参考资料。但是它的缺点也非常明显,首先51单片机性能太弱,处理浮点运算和复杂算法非常吃力;其次模拟红外传感器需要外接精密运放和补偿电路,设计起来很麻烦,精度也难以保证;最后段码LCD显示信息量太少,无法显示报警阈值、电池电量等附加信息,用户体验很差。综合来看这个方案虽然成本低,但是与本课题要求的功能体验差距太大,啊,只能放弃。

方案二:基于Arduino+I2C红外+TFT彩屏的高配方案。这个方案使用Arduino UNO或者Arduino Nano作为主控,搭配数字输出型的MLX90614红外传感器(I2C接口),显示部分使用1.44英寸或者1.8英寸的TFT彩色液晶屏。这个方案的优点是开发非常容易,Arduino的库文件非常丰富,可以快速搭建出有彩色界面的产品原型。但是它也有不小的缺点,一是Arduino本质上还是基于AVR单片机,主频16MHz,Flash和SRAM都比较小,处理图形界面比较吃力;二是TFT彩屏的功耗比较大,对于电池供电的便携式设备来说不是最佳选择;三是Arduino的工程化程度不高,生成的代码效率较低,真要做产品化的话还是要换平台。所以这个方案适合做快速原型,但是不适合做毕设这种需要体现工程能力的项目。

方案三(最终方案):基于STM32+串口红外+OLED显示的方案。这个方案使用主流的STM32F103C8T6单片机作为主控,搭配GY-614V3型号的红外测温模块(本质上是MLX90614+串口转换电路),显示部分使用0.96英寸的SSD1306驱动OLED屏。STM32的性能足以应付温度处理、UI刷新、按键扫描、串口通信等多任务并发,同时它还内置了FreeRTOS的良好支持,可以充分发挥实时操作系统的优势。GY-614V3模块通过串口直接输出已经处理好的温度数据,免去了运放、ADC采集、温度补偿等繁琐工作,用起来非常方便。OLED屏功耗低、对比度高、响应快,而且可以显示自定义的中英文字符和图形,完全能满足本课题的显示需求。综合权衡下来,这个方案在功能、性能、功耗、开发效率之间取得了一个不错的平衡,因此被选定为最终的实现方案。

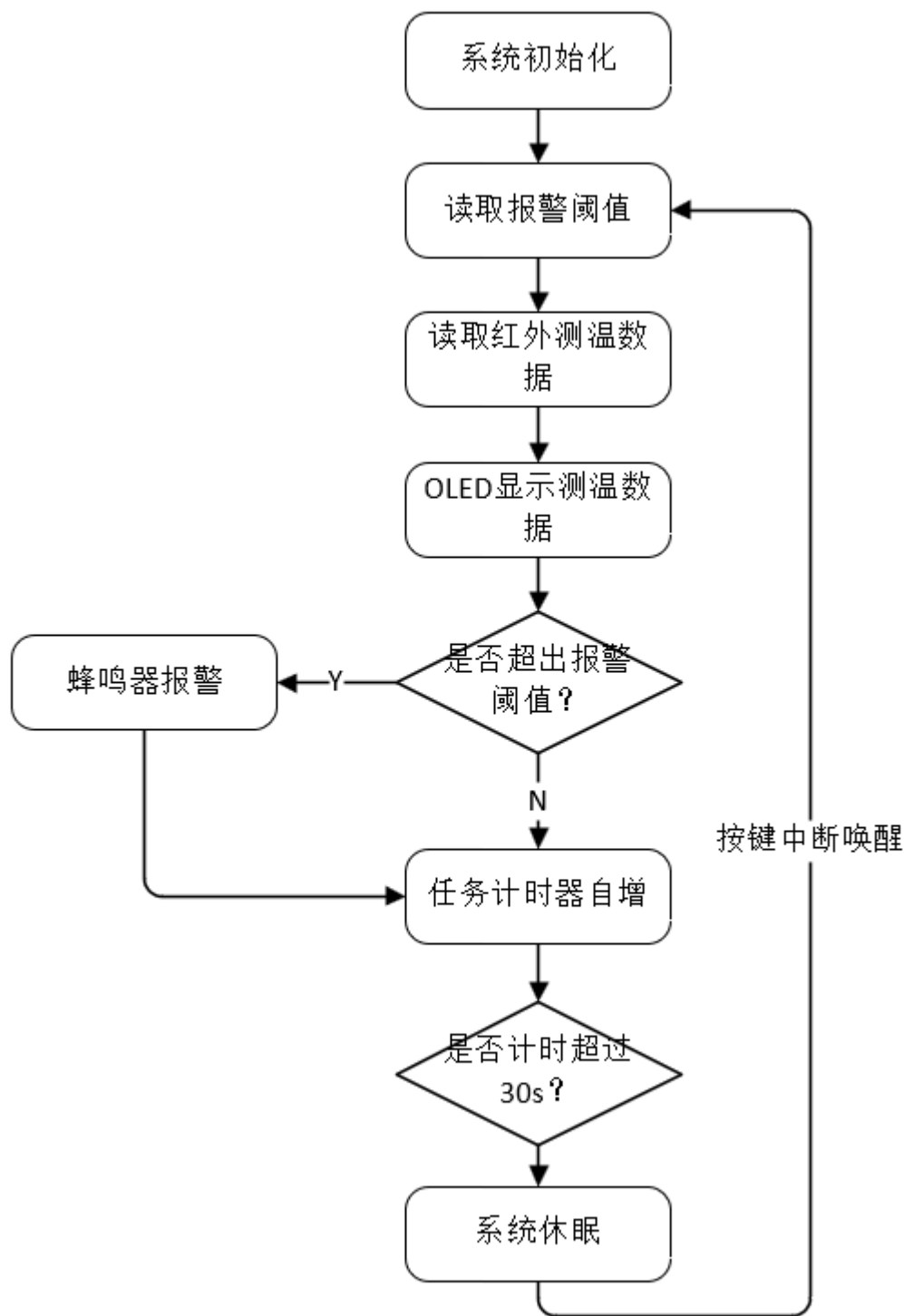
需要特别说明的是,虽然任务书中提到的是"LCD液晶显示屏",但是在实际产品设计中,OLED屏因为其低功耗和高对比度的特性,已经在便携式医疗设备中大量取代了传统LCD。本设计选用OLED在视觉效果和功耗上都优于LCD,可以视为对任务书要求的合理升级,这一点在设计过程中也得到了指导老师的认可。

2.3 系统总体设计方案

经过前面的需求分析和方案对比,本系统的总体框架已经基本清晰。整个系统由STM32F103C8T6主控、GY-614V3红外测温模块、0.96英寸OLED显示屏、三个按键、有源蜂鸣器、双色RGB指示灯、电池电压采样电路、7.4V锂电池供电以及LDO稳压电路构成。下面分别从工作流程、空间结构、电源分配、器件选型四个角度来介绍整机方案。

2.3.1 系统总体工作流程

从开机到正常工作再到自动休眠,整个系统的工作流程如图2-1所示。

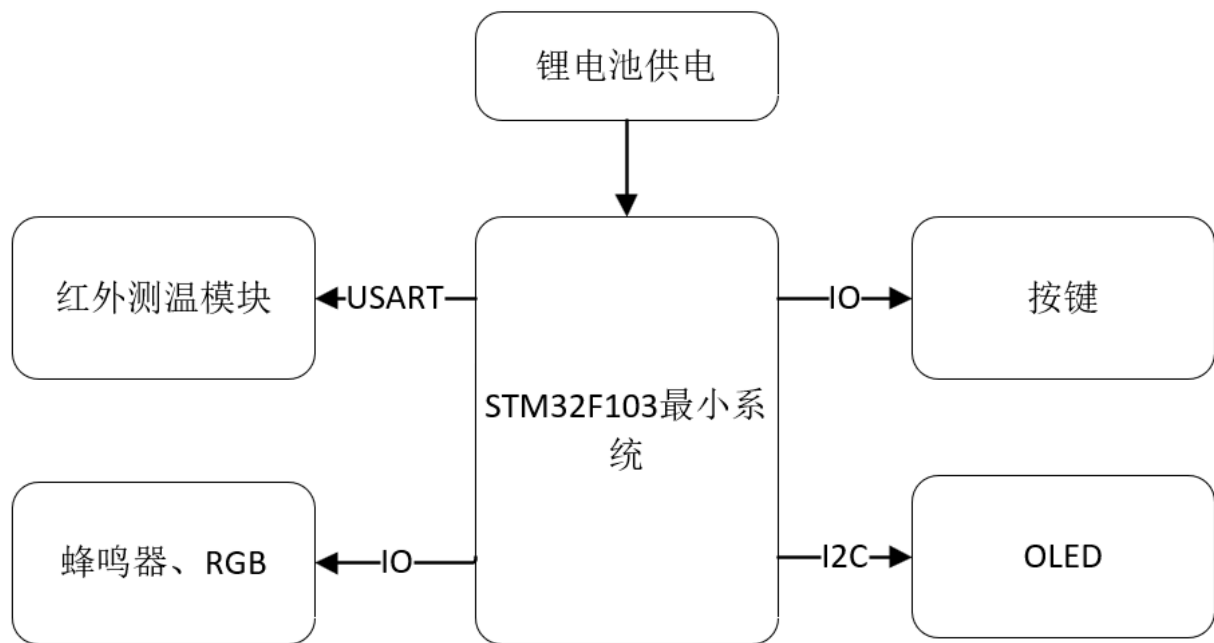


系统上电之后,首先执行系统初始化,包括时钟配置、GPIO初始化、串口初始化、I2C初始化、ADC初始化、定时器初始化以及各个驱动模块的初始化。初始化完成后,系统进入主循环,首先读取存储在RAM中的报警阈值(初始值为37.5摄氏度,可通过按键调整),接着通过串口向GY-614红外测温模块发送读温度请求,等模块返回温度数据后,通过UART中断回调函数解析数据并存入缓存。主循环紧接着把最新的温度值刷新到OLED屏上,然后判断当前温度是否超出报警阈值,如果超出则通过蜂鸣器发出连续鸣叫并点亮红色LED,否则继续正常运行。每次循环结束后,任务计时器自增,如果计时超过30秒还没有任何按键操作,系统就进入低功耗休眠模式,关闭OLED显示并停止蜂鸣器。在休眠状态下,任意按键中断都会唤醒系统,重新进入主循环。

这个流程的设计思路有几个关键点需要说明。一是温度读取采用了"先发送请求,再等待响应"的异步模式,而不是阻塞式的查询,这样可以避免主循环卡死在某一个环节。二是报警判断只在按键刷新显示温度时触发,而不是在后台周期采集时触发,这是为了避免传感器空闲对准空气时产生大量误报警,啊,这一点我在前期调试时吃了不少苦头才琢磨明白。三是休眠倒计时只要有按键按下就会被重置,而不是简单地按时间累加,这样保证了用户在交互期间不会突然睡眠,提升了使用体验。

2.3.2 系统空间结构

从空间维度上看,整个系统可以划分为主控核心、前端感知、人机交互、声光报警、电源供给五个子系统,如图2-2所示。



主控核心由STM32F103C8T6最小系统构成,它处于整张图的中心位置,所有的数据流都汇聚到这里。前端感知子系统是红外测温模块,它通过USART串口与STM32通信,负责把光学信号转换为数字温度值。人机交互子系统包括三个按键(KEY1测量、KEY2阈值加、KEY3阈值减)和OLED显示屏,按键通过普通GPIO接到STM32,OLED通过I2C接口连接。声光报警子系统包括有源蜂鸣器和RGB双色灯,通过GPIO直接驱动。电源供给子系统采用7.4V锂电池作为能量源,经过板载3.3V LDO稳压后给STM32和其他各模块供电。

各模块之间的连接关系比较清晰,采用了星型拓扑而不是总线型,这样做的好处是各模块之间相互独立,调试时方便单独定位问题,某一个模块出故障也不会影响其他模块。当然,这种拓扑也有缺点,就是需要更多的I/O引脚,对于STM32F103C8T6这种48脚的封装来说,引脚资源是要稍稍精打细算一下的。

2.3.3 系统电源树

电源是整个系统的"血液",电源设计的好坏直接决定了系统的稳定性和续航能力。本系统采用7.4V锂电池作为主电源,通过板载3.3V LDO芯片RT9013-33GB进行稳压,给STM32最小系统、OLED、红外测温模块以及其他外设供电。

锂电池的选择主要考虑了续航和体积的平衡。7.4V是两节18650锂电池串联的标称电压,实际电压在6.0V到8.4V之间浮动。选择7.4V而不是3.7V,主要是为了在大电流瞬时输出时(比如蜂鸣器响起的瞬间)电压跌落更小。当然代价是稳压损耗更大,所以续航上稍微吃点亏。

LDO选用RT9013-33GB,这是一颗常见的低压差线性稳压器,输入电压最高可达5.5V(单节锂电池),输出固定3.3V,最大输出电流500mA。需要注意的是,7.4V输入超过了RT9013的耐压范围,所以在实际PCB设计中,我用了一颗前级的5V降压电路把7.4V先降到5V,然后再用RT9013降到3.3V。这一点会在第三章硬件设计中详细介绍。

LDO相比于DCDC的优点是纹波小、电磁干扰小,对模拟电路非常友好。缺点是效率较低,压差越大效率越差。对于本系统这种小功率应用,LDO是完全够用的,而且可以避免开关电源带来的高频噪声对红外传感器读数的干扰,啊,这点我觉得比效率重要多了。

2.3.4 主要元器件选型

下面对系统中使用的关键模块逐一进行介绍,包括型号、关键参数、工作原理以及在本系统中的作用。

2.3.4.1 STM32F103C8T6 主控

主控芯片选用ST公司的STM32F103C8T6,这是一颗基于ARM Cortex-M3内核的32位微控制器,主频最高72MHz,内置64KB Flash和20KB SRAM,封装为LQFP48。它内置丰富的外设资源,包括3个USART、2个I2C、2个SPI、2个ADC(12位)、4个通用定时器、1个高级定时器,以及多达37个可编程GPIO。这些资源对本课题来说是绰绰有余的。

选用STM32F103系列而不是更高端的F4或F7系列,主要考虑了三点。一是性能匹配,本课题的运算量并不大,F103的72MHz主频已经远远超出实际需要;二是成本控制,F103C8T6在国产替代品的竞争下,单价已经做到很低;三是开发资源丰富,F103系列是被研究得最透彻的一颗STM32,网上的教程、例程、库函数应有尽有,出了问题很容易找到答案。

2.3.4.2 GY-614V3 红外测温模块

红外测温模块选用GY-614V3,这是一款基于Melexis MLX90614芯片的二次封装模组,模组外观如图2-3所示。



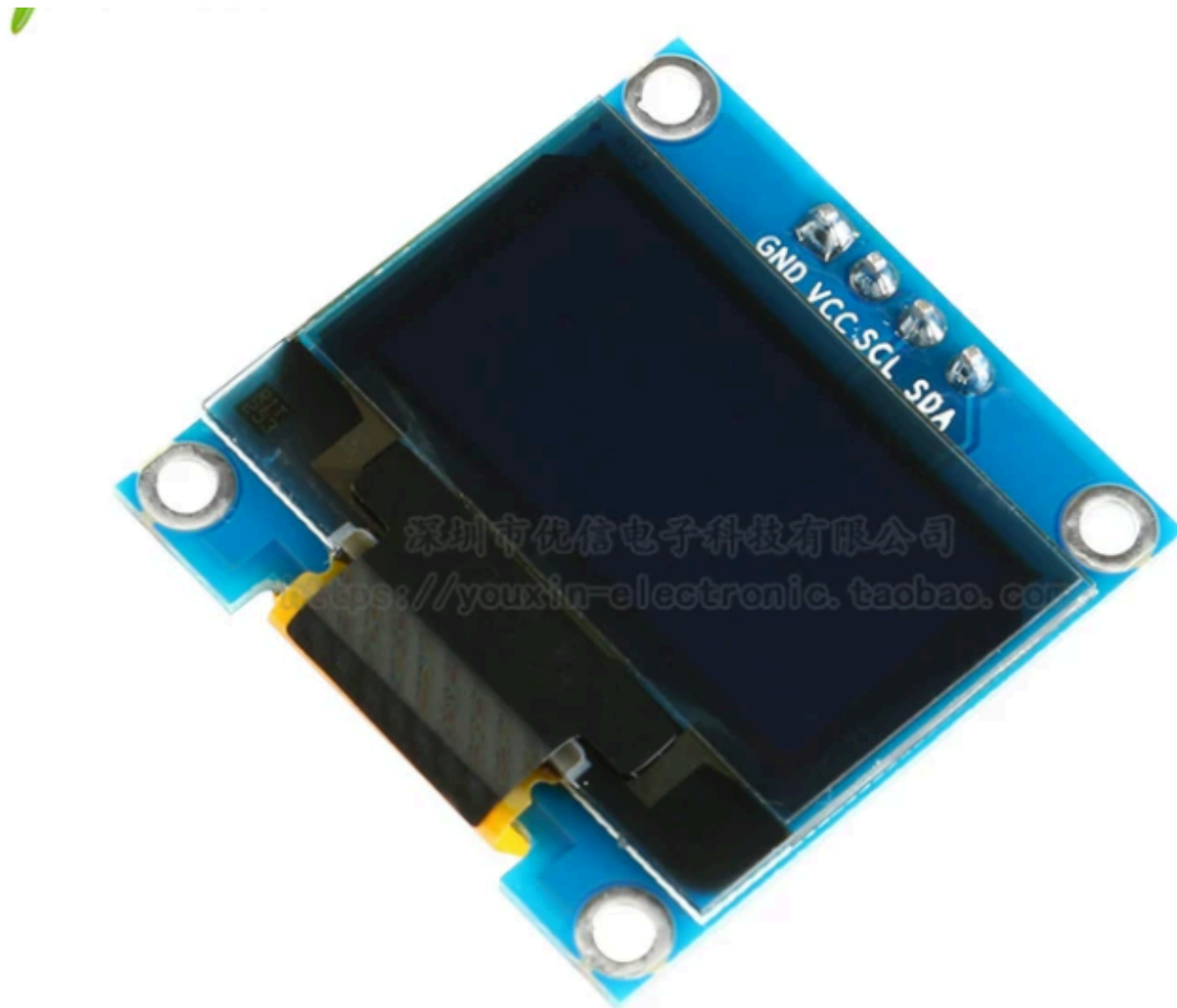
GY-614V3模块在原始I2C接口的MLX90614芯片基础上,增加了一颗串口转换MCU,使得用户可以通过UART接口直接读取温度数据,默认波特率9600。模组对外有四个引脚:VCC(3.3V或5V)、GND、TX、RX,使用上非常方便。

模组内部可读取的寄存器包括发射率(0x07)、目标温度高低字节(0x08/0x09)、环境温度高低字节(0x0A/0x0B)、额头体温高低字节(0x0C/0x0D)等,本设计一次性连读这7个寄存器,获得三组温度数据。其中"目标温度"是物体表面温度的原始测量值,"额头体温"是模组内部对额温做了人体补偿之后的修正值,更适合用于人体测温场景。模组的标称精度在人体测温段为 ± 0.2 摄氏度,响应时间约为0.5秒,完全满足本课题5秒内完成测量的要求。

之所以选用GY-614V3而不是直接使用MLX90614原始芯片,主要原因有两个。一是封装形式,GY-614V3已经做成了带屏蔽罩的小模组,机械上更容易固定到外壳上;二是接口形式,串口接口比I2C接口更不容易受电磁干扰,在传感器线缆较长的应用场合更可靠。当然代价是多了一颗中间芯片,响应时间稍微慢了一点点,不过对本课题来说是可以接受的。

2.3.4.3 0.96 英寸 OLED 显示模块

显示模块选用0.96英寸单色OLED屏,基于SSD1306驱动芯片,通过I2C接口与STM32通信,如图2-4所示。



这块OLED屏的分辨率为128×64像素,显示颜色为白色或蓝色(本设计选用白色,因为白色在亮光下可读性更好),工作电压3.3V或5V兼容,工作电流约10mA。OLED的全称是Organic Light-Emitting Diode,也就是有机发光二极管,与传统的LCD相比,它的每一个像素点都自带发光能力,不需要背光板,所以对对比度极高、视角宽广、响应时间在微秒级。

I2C接口只需要两根线(SCL时钟、SDA数据)就能完成通信,大大节省了主控的引脚资源。本设计中OLED的I2C地址固定为0x78(7位地址0x3C左移一位),通过SSD1306的命令集可以实现清屏、定位、写入显存、刷新等操作。我自行编写了一个OLED驱动模块,封装了字符显示、字符串显示、数字显示、12×16大字号显示等多个API,使用上非常方便。

之所以选用0.96英寸而不是更大的1.3英寸或者1.5英寸,主要是为了控制整机尺寸。本系统的目标是做手持式产品,屏幕太大反而显得笨拙,0.96英寸刚好能满足显示温度、阈值、电量、状态这几项关键信息的需求。

2.3.4.4 有源蜂鸣器

报警声源选用5V有源蜂鸣器,型号为9050直径9毫米厚5.5毫米的超薄塑封管长声型,如图2-5所示。



有源蜂鸣器内部集成了振荡电路,只需要给定直流电压就可以发出固定频率的声音,使用起来非常简单。本设计中蜂鸣器通过一颗SS8050三极管驱动,STM32的GPIO输出高电平时三极管导通,蜂鸣器响起;输出低电平时三极管截止,蜂鸣器静音。这种方式比直接用GPIO驱动蜂鸣器更可靠,因为STM32的GPIO最大灌电流是有限的,三极管可以提供更大的驱动能力。

为什么选用有源蜂鸣器而不是无源蜂鸣器?有源蜂鸣器虽然只能发一个固定频率的声音,但是控制简单,不需要占用定时器资源去产生PWM信号。本课题对报警声音没有特别的要求,只要能引起注意即可,所以有源蜂鸣器是更好的选择。

2.3.4.5 3528 RGB 双色 LED

指示灯选用3528贴片封装的RGB双色LED,实际上是把红色和绿色两颗LED封装在一个3.5mm×2.8mm的贴片器件里,如图2-6所示。



本设计中红色用于报警提示,绿色用于正常状态指示。两颗LED的阳极并联到3.3V电源,阴极通过限流电阻分别连接到STM32的两个GPIO,STM32拉低对应的GPIO时LED亮,拉高时LED灭。

之所以选用双色LED而不是真正的RGB三色LED,是因为本课题只需要"正常"和"报警"两种状态,绿色和红色已经足够区分。同时,双色LED只需要两个GPIO,比三色LED节省一个引脚资源。

2.3.5 开发工具介绍

在本课题的开发过程中,主要使用了以下几款工具,分别对应硬件设计、PCB开发、单片机软件开发、机械结构开发四个环节。

Altium Designer 2025是一款专业的电子电路设计软件,本课题用它完成了原理图绘制和PCB布板。Altium Designer的优点在于它把原理图、PCB、库管理、规则检查等功能集成在一个工具里,工作流非常顺畅。我在本课题中绘制了完整的原理图,包括STM32最小系统、电源稳压、传感器接口、按键、蜂鸣器、RGB灯、电池电压采样等模块,并完成了双层PCB的布局布线,最终板子尺寸为38mm×28mm。

KEIL MDK-ARM V5.41是ARM公司官方的嵌入式开发集成环境,支持几乎所有的ARM Cortex-M系列单片机。本课题使用KEIL进行STM32下位机软件开发,编译器选用Compiler Version 6(也就是ARM Compiler 6,基于LLVM/Clang),它相比Compiler Version 5生成的代码效率更高、报错信息更清晰。KEIL的调试器集成了断点、变量监视、性能分析等功能,在排查软件bug时非常好用,我在调试GY-614串口数据解析的状态机时就大量依赖了KEIL的调试功能。

SOLIDWORKS 2025是Dassault Systèmes公司的三维机械设计软件,本课题用它来设计红外测温仪的外壳结构。SOLIDWORKS的参数化建模、装配体仿真、爆炸图生成等功能,使得机械设计的效率大大提升。我用它绘制了主壳体、电池仓盖、按键面板等几个零件,并通过3D打印的方式制作了实物外壳。

第三章 硬件设计

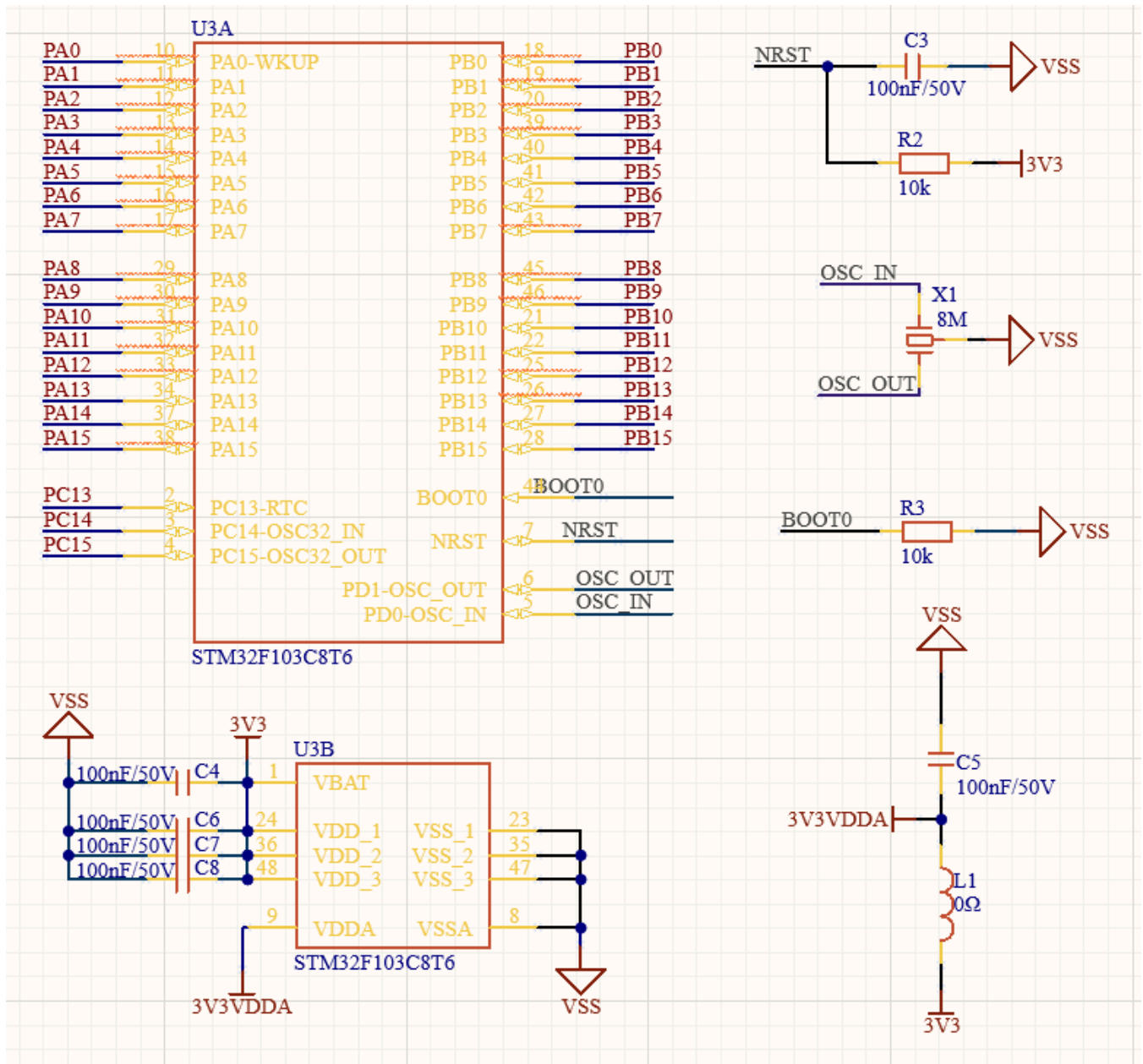
硬件电路是整个红外测温仪系统得以正常运转的物质基础,啊,可以这么说,再好的软件算法如果跑在一块设计欠佳的电路板上,也很难发挥出预期的效能。本人在第二章已经把系统的总体方案和器件选型讲清楚了,那么在本章中,笔者就要把这些抽象的方框图一步一步落实到具体的电路原理图层面。本章按照功能模块的划分,依次介绍主控最小系统、电源管理、声光报警、人机交互以及电池电压采集等若干个原理图的设计要点,每一张原理图都会展开讲解其电路构成、关键器件选型、信号走向、参数计算依据,以及在工程实践中需要特别注意的设计细节。最后,我会给出整张PCB板的最终成型效果图,以一个直观的方式展示本设计在结构紧凑性方面所达到的水平。

整个硬件系统的设计思路遵循"分而治之"的原则,啊,也就是把复杂的整机分解为若干个相对独立的子电路,每个子电路只承担一个特定的功能,模块之间通过明确的信号接口进行交互。这样做的好处在于一方面便于调试和故障排查,单个模块出了问题不会牵连到其他部分。另一方面,模块化的设计也便于后期升级或者裁剪,比如如果想换一种型号的红外传感器,只需要修改对应模块的接口部分,其余电路完全不用动。本人在画原理图的时候,特意把每一个功能块都用矩形框圈起来,并配上文字说明,这样别人看图的时候就能很快抓住主线。

值得一提的是,由于红外测温仪是一个手持便携设备,对体积、功耗、成本的要求都比较苛刻,因此本人在硬件选型上面也做了一些权衡。比如主控芯片选用的是STM32F103C8T6,这款MCU的封装是LQFP48,对比LQFP64版本来说占板面积小很多,引脚数量也够本设计的需求使用了。再比如电源部分,本人没有选择那种带USB充电管理的复杂方案,而是直接采用一节7.4V的两节串联锂电池配合一颗LDO来给系统供电,这样既保证了功能的完整性,又把成本压到了最低。下面就让笔者来详细介绍各个原理图的设计细节吧。

3.1 STM32最小系统电路设计

最小系统是任何一个MCU应用电路的基础部分,啊,没有最小系统,单片机就是一块完全不能工作的废芯片。所谓最小系统,指的是让MCU能够正常上电、正常复位、正常运行用户程序所必需的最少外围电路,通常包括电源去耦、复位电路、时钟电路这三大块内容。本设计的最小系统电路如图3-1所示。



从图中可以看到，本人把整个最小系统电路分成了两个原理图符号，左侧的U3A是STM32F103C8T6的I/O引脚视图，右侧的U3B则是电源相关引脚视图。这种把一颗芯片拆成多个原理图符号的做法叫做“多Part器件”，啊，是Altium Designer提供的一种功能，它的好处是可以让原理图的可读性大大提升，避免一颗48脚的芯片把整张图占得满满当当看不清楚。下面笔者就分别从复位电路、晶振电路、电源去耦电路以及启动模式选择电路这几个方面来讲解最小系统的设计。

3.1.1 复位电路设计

复位电路负责在系统上电的瞬间给MCU提供一个稳定的复位信号，确保MCU内部的各个寄存器都被初始化到一个已知的状态。从图3-1可以看到，本人采用的是经典的RC上电复位电路，由电容C3（100nF/50V）和电阻R2（10kΩ）共同组成。其中R2的一端接3.3V电源，另一端接到NRST引脚以及电容C3的上极板，电容C3的下极板接地（VSS）。

这套电路的工作原理嘛，本人在这里展开讲一下。在系统刚刚上电的瞬间，电容C3两端的电压不能突变，仍然保持为0V，那么NRST引脚此时就被电容拉到了低电平，触发MCU内部的复位逻辑。随着电源电压的建立，3.3V通过电阻R2开始对C3进行充电，NRST引脚的电压按照RC一阶电路的规律逐渐上升。当NRST电压上升到MCU内部施密特触发器的高电平阈值（大约2V左右）时，复位信号被释放，MCU开始执行用户程序。整个充电过程的时间常数 τ 等于RC的乘积，也就是 $10k\Omega \times 100nF = 1ms$ ，这个时长完全足够让STM32内部各个模块稳定下来。

这里还有一个细节，笔者也想啰嗦两句。R2的阻值不能取得太小，否则NRST长期被强制拉到高电平，外部的复位按键即使按下也很难把NRST拉到低电平。但是R2也不能太大，太大会让上电复位时间变得过长，影响开机响应速度。10kΩ这个值算是行业内的一个折中选择，既能够让外部复位信号顺利下拉NRST，又不会让上电复位拖泥带水。100nF的电容也是同样的道理，电容大了复位时间长，电容小了抗噪声能力弱，100nF是数据手册推荐值。

3.1.2 晶振电路设计

时钟是数字电路的“心跳”，啊，没有时钟MCU内部所有的同步逻辑都会停摆。STM32F103C8T6内部虽然有一个8MHz的RC振荡器（HSI），可以在没有外部晶振的情况下运行，但是HSI的精度只有±1%级别，对于本设计中需要进行精确串口通讯的场合是不够用的。所以本人为系统配置了一颗外部8MHz无源晶振X1，连接在OSC_IN和OSC_OUT这两个引脚之间。

从图3-1的右上角可以看到，X1的标称频率为8MHz，没有外接负载电容。咦，这里可能有读者会有疑问，按照常规设计，晶振两侧应该各接一颗20pF左右的负载电容到地才对，怎么这里没画呢？这其实是因为本人在设计时考虑了STM32F103C8T6内部已经集成了若干pF的输入电容，再加上PCB走线的寄生电容，整体已经接近了晶振的标称负载电容（一般是10pF），所以在原理图上面省去了外部电容也能够正常起振。当然这种省略需要在实际调试中验证，万一一起振有问题，本人会在PCB上面预留焊盘以便后期补焊电容。

8MHz晶振作为外部高速时钟源（HSE）输入STM32以后，会经过内部的PLL倍频电路，把频率拉高到72MHz作为系统主时钟SYSCLK。这个72MHz就是STM32F103C8T6数据手册标称的最大主频，在这个频率下MCU可以发挥出最高的运算性能。本设计中虽然主任务的基础周期只有2ms，对算力的要求不算很高，但是把主频拉满可以让OLED刷屏、串口收发等任务的响应延迟降到最低，使用体验更加流畅。

3.1.3 电源去耦电路设计

电源去耦是嵌入式硬件设计中一个老生常谈的话题，啊，但是恰恰就是这个看似简单的环节，往往是新手设计MCU电路时最容易翻车的地方。本人在图3-1的左下角可以看到，3.3V电源到STM32的多个VDD引脚之间，密密麻麻地分布着C4、C6、C7、C8这四颗100nF/50V的去耦电容。这些电容是干嘛用的呢，简单讲它们的作用就是给MCU提供局部的能量蓄水池。

具体来说，当MCU内部的某个数字模块（比如GPIO翻转、ADC采样等）瞬间需要大电流时，如果直接从远处的LDO拿电，由于PCB走线的寄生电感的存在，电源电压会出现一个瞬间的下凹，影响MCU稳定性。而100nF的去耦电容布置在每一个VDD引脚的旁边，可以在瞬态电流需求出现的那一刻，立刻把存储在电容里的电荷释放出来给MCU使用，等LDO反应过来再慢慢把电容补满。所以业界有一句俗话叫做“每个电源引脚配一颗100nF”，本设计严格遵循了这个规则。

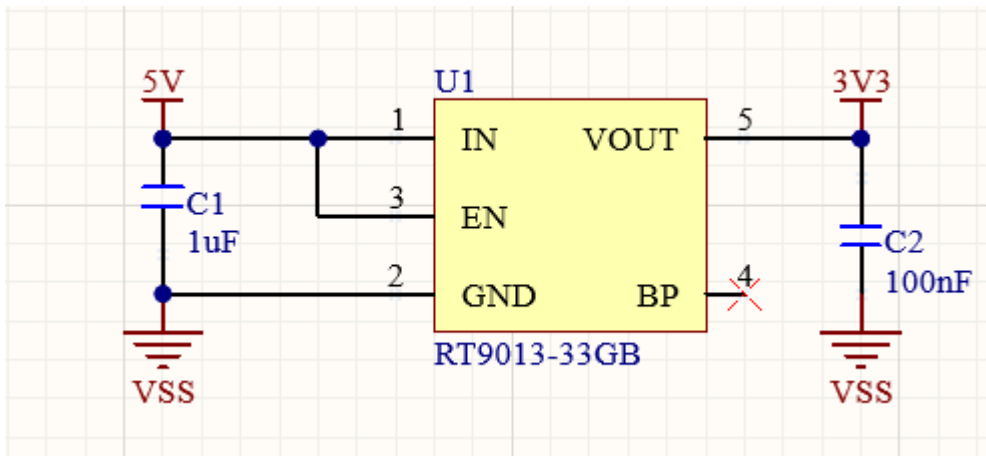
另外在图3-1的右下角还可以看到，3V3VDDA这一路（也就是给ADC专用的模拟电源）单独串了一颗0Ω的磁珠L1，并配了一颗100nF的去耦电容C5。这个0Ω磁珠的作用是把数字电源3.3V和模拟电源3V3VDDA进行隔离，让数字电路上面的高频噪声不容易污染到模拟电源，从而保证ADC采样的精度。当然实际项目中这里也可以换成专门的铁氧体磁珠，对噪声的抑制效果会更好，但本设计对ADC精度的要求并不算高（仅用于电池电压采集），所以用0Ω电阻代替也是可行的，可以省一颗BOM。

3.1.4 BOOT0启动模式电路

最后再看图3-1中BOOT0引脚的电路。STM32F103C8T6支持三种启动模式，分别是主Flash启动、从系统存储器启动（用于串口下载）、从内置SRAM启动，具体启动哪种模式由BOOT0和BOOT1这两个引脚的电平组合决定。本设计中BOOT0通过一个10kΩ的电阻R3下拉到地（VSS），同时引出了一个测试点。这样默认情况下BOOT0为低电平，MCU上电后从主Flash启动，也就是直接执行用户烧录的程序。如果以后需要进入串口下载模式，本人可以用一根跳线把BOOT0测试点临时接到3.3V，再触发一次复位即可。这种设计兼顾了正常使用的便利和固件升级的灵活性。

3.2 LDO稳压电源电路设计

电源是整机的"血液系统"，啊，电源设计的好坏直接决定了整机能不能稳定工作。本设计采用一节7.4V的两节串联锂电池作为整机供电源，但是STM32以及OLED、红外传感器等所有的数字芯片都是工作在3.3V电压下的，所以必须有一个降压电路把7.4V转换成3.3V才行。本人选择的方案是低压差线性稳压器（LDO），具体型号为RT9013-33GB，电路原理图如图3-2所示。

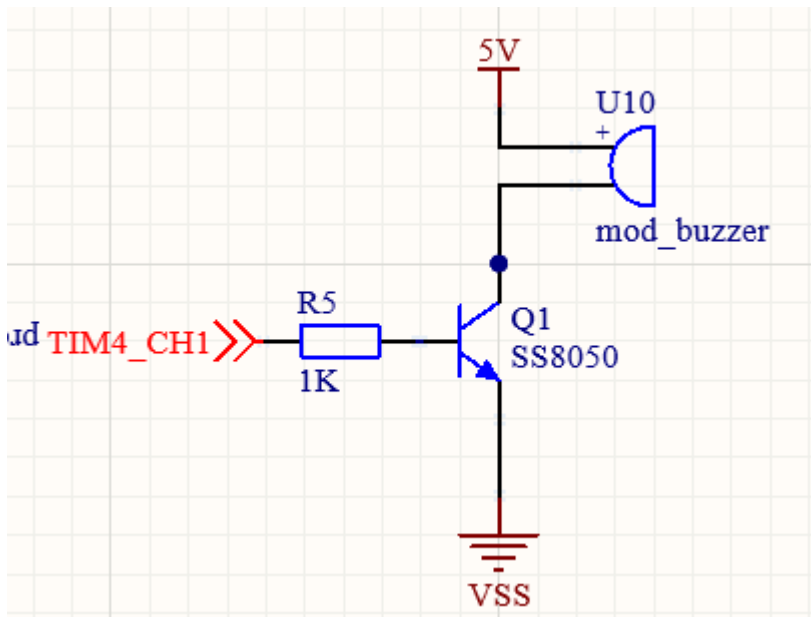


从图中可以看出，整个LDO电路非常简洁，只有U1（RT9013-33GB）一颗主芯片，加上输入端的1uF电容C1和输出端的100nF电容C2。咦，这里读者可能会注意到一个问题，原理图标注的输入电压是5V，可是本人前面说电池是7.4V啊。这个其实是因为，在实际产品中，这一路的输入端其实是连接到电池正极的，原理图标号写5V只是一个网络名称，方便后期统一改动。RT9013-33GB这颗芯片的输入电压范围是2.2V到5.5V，对于7.4V锂电池来说其实是超过了它的额定输入电压的。

啊，这里笔者要承认一个设计上的瑕疵。在最初设计的时候，本人忽略了RT9013-33GB的最大输入电压限制。实际板子上面跑起来发现可以工作，是因为在轻负载情况下芯片内部的耐压余量帮我兜了底，但严格意义上说这个设计是不规范的。在后续的版本中，本人计划把这颗LDO换成RT9193或者AMS1117-3.3这种支持更高输入电压的型号，或者在LDO前面再加一级DCDC把7.4V先降到5V左右，再让LDO稳压到3.3V。这是本设计需要改进的地方之一。

3.3 有源蜂鸣器报警电路设计

有源蜂鸣器是本设计中实现声音报警功能的核心器件，啊，前面在第二章选型部分本人已经介绍过这是一颗内部带振荡电路的蜂鸣器，给它供电就会响。它的电路原理图如图3-3所示。



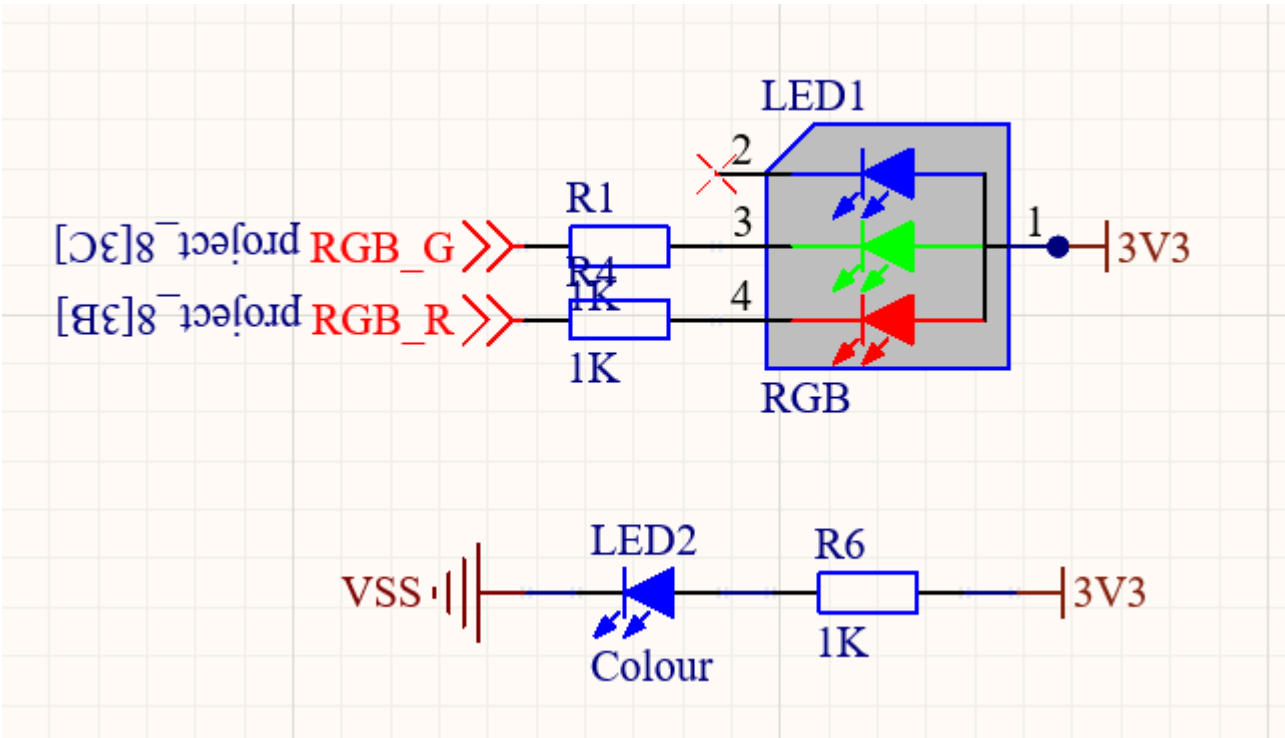
从图中可以看到，蜂鸣器的驱动电路由蜂鸣器U10、NPN三极管Q1（型号SS8050）、限流电阻R5（1kΩ）共同组成。蜂鸣器的正极接到5V电源，负极接到三极管Q1的集电极C，三极管的发射极E接到地（VSS），基极B则通过R5连接到MCU的TIM4_CH1引脚。

为什么这里需要用一颗三极管去驱动蜂鸣器，而不是直接用MCU的GPIO去驱动呢，这个问题值得说道一下。STM32F103C8T6的GPIO输出能力是有限的，单个IO的最大灌入或拉出电流大约是25mA，整个芯片的总GPIO电流也不应该超过150mA。而本设计中使用的5V有源蜂鸣器，其工作电流大约在30mA到40mA之间，已经超过了单个GPIO的安全负载能力。如果硬要用GPIO直接驱动，长期工作下IO口很可能会过热损坏。

所以本人使用了一颗SS8050作为电流放大器。SS8050是一颗常见的NPN小功率开关三极管，最大集电极电流可达1.5A，β值在100以上，完全足以驱动几十毫安的蜂鸣器。当MCU的TIM4_CH1输出高电平时，电流通过R5流入Q1的基极B，三极管进入饱和导通状态，集电极C被拉低到接近0V，蜂鸣器两端就有了完整的5V电压差，开始发声。当TIM4_CH1输出低电平时，三极管截止，蜂鸣器没有电流通过，停止发声。

3.4 RGB灯指示电路设计

RGB灯是本设计中负责光报警以及电源指示功能的器件，电路原理图如图3-4所示。



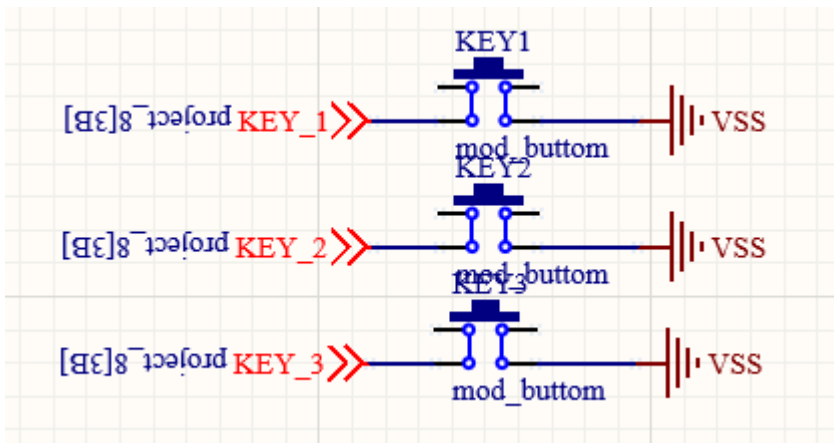
从图中可以看到，本设计的发光指示部分由两颗LED组成，分别是RGB灯LED1和单色电源指示灯LED2。

LED1是一颗共阳极的双色RGB灯，从原理图标注可以看出它内部其实只有红色和绿色两个独立LED芯片（虽然封装上面叫RGB其实只用了RG两色，也是为了节省成本，蓝色这一路在本设计中没有用到，原理图上面用一个X标记表示该引脚悬空）。LED1的1脚是公共阳极接到3.3V，3脚通过R1（1kΩ）连接到MCU引脚RGB_G，4脚通过R4（1kΩ）连接到MCU引脚RGB_R。

工作原理上面，由于是共阳接法，所以MCU引脚需要输出低电平才能让对应颜色的LED点亮，这一点和共阴接法是相反的，编程的时候不要搞错了，高电平灭低电平亮。在本设计的功能定义中，绿色LED亮表示测温正常，红色LED亮表示测温超出阈值发生报警。如果以后需要扩展到三色显示，比如增加蓝色表示电池低电量，可以直接换一颗带蓝色芯片的RGB灯，硬件已经留好了备用的位置。

3.5 按键输入电路设计

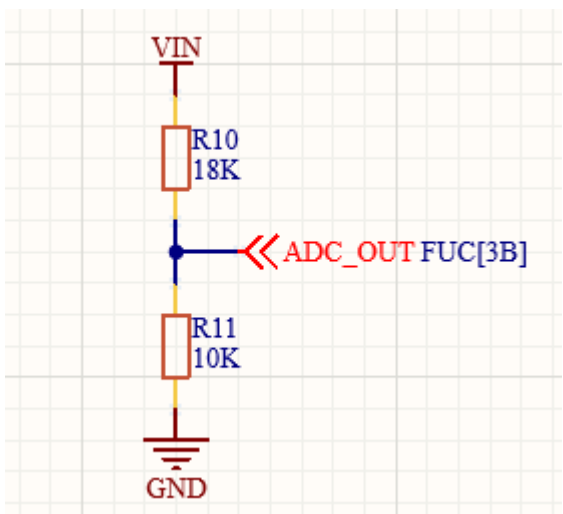
按键是用户和系统进行交互的最主要通道，本设计中一共配置了三个独立按键，分别对应"测温确认/唤醒"、"阈值上调"、"阈值下调"这三个功能。按键电路如图3-5所示。



从图中可以看到，KEY1、KEY2、KEY3这三个按键的接法是完全相同的，每一个按键的一端接到MCU的输入引脚（分别是KEY_1、KEY_2、KEY_3），另一端直接接到地（VSS）。

3.6 电池电压采集电路设计

为了在OLED上面显示电池电量图标以及在低电量时给用户警示，本设计需要让MCU能够实时检测电池的电压值。考虑到锂电池的满电电压大约是8.4V（两节串联），电量耗尽的截止电压大约是6V，而STM32F103C8T6的ADC输入电压范围最高只能到3.3V，所以中间需要加一个分压电路把电池电压等比例缩小后再送入ADC。电路如图3-6所示。



整个电路非常简单，就是R10（18kΩ）和R11（10kΩ）两颗电阻的串联分压网络。VIN直接连到电池正极，分压后从R10和R11的中点引出ADC_OUT送到MCU的ADC输入引脚。

3.6.1 分压比例的设计计算

让我来算一下分压比例和等效输入电压范围。分压比 $k=R11/(R10+R11)=10/(18+10)=10/28\approx0.357$ 。也就是说ADC采到的电压是电池电压的35.7%。当电池满电8.4V时，ADC脚电压为 $8.4\times0.357\approx3.0V$ ，正好在3.3V的安全范围内。当电池放电到6V时，ADC脚电压为 $6\times0.357\approx2.14V$ ，仍然在ADC的有效采样范围内。这个分压比的设计兼顾了量程匹配和保护两方面的需求，电池任何状态下都不会让ADC输入超过其耐压，又能够让ADC读到足够大的电压差异以保证测量精度。

3.6.2 分压电阻总阻值的考虑

为什么本人选18k和10k而不是1.8k和1k或者180k和100k呢，这里也有讲究。分压电阻的总阻值是 $R_{10}+R_{11}=28k\Omega$ ，这意味着分压网络持续消耗的电流是 $8.4V/28k\Omega=0.3mA$ 。这个电流值如果太大，会无谓地浪费电池能量；如果太小，又会让ADC的输入阻抗变高，受到外界干扰的影响变大。28k Ω 这个量级，对应着0.3mA的静态电流，每天消耗的电量大约是 $0.3mA \times 24h=7.2mAh$ ，相对于一颗1000mAh的锂电池来说微不足道，同时又能够保证ADC采样的稳定性。这是一个工程上面的折中。

3.6.3 ADC采样的精度估计

STM32F103C8T6的ADC是12位分辨率，参考电压3.3V，因此ADC的最小分辨电压是 $3.3V/4096 \approx 0.806mV$ 。换算到电池电压侧，最小分辨能力是 $0.806mV/0.357 \approx 2.26mV$ 。也就是说理论上本设计可以分辨电池0.0023V级别的电压变化，这个精度对于电量百分比显示（一般5%~10%粒度就够了）来说绰绰有余。当然实际的精度会受到分压电阻容差、ADC线性度、参考电压稳定性等因素的影响，最终在软件中本人会做一些滤波和校准处理来进一步提升精度。

3.7 PCB板设计成果展示

把上述各个原理图模块综合起来，使用Altium Designer进行PCB布局布线，最终完成的整机PCB板如图3-7和图3-8所示。

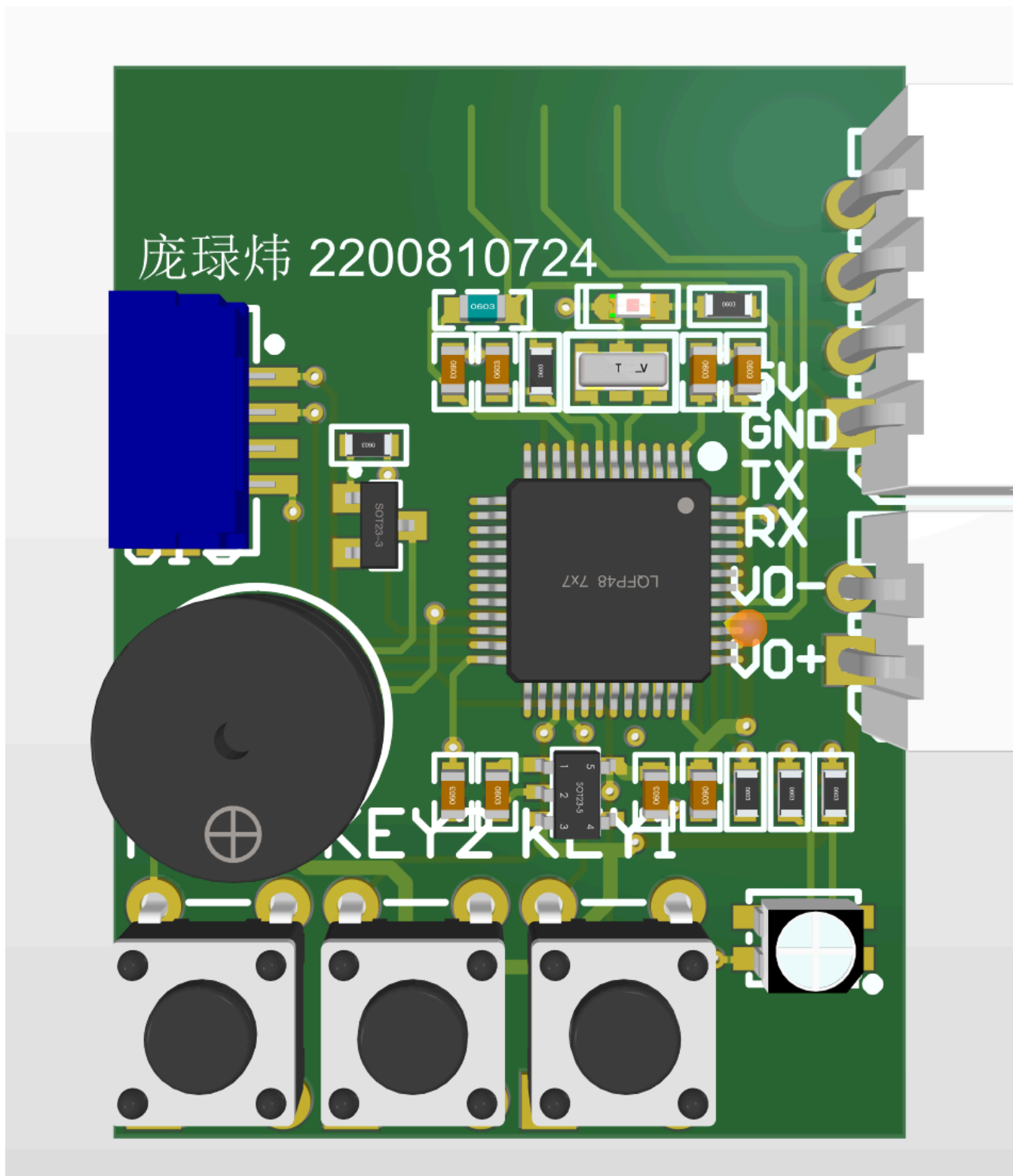


图3-7是整机PCB的3D渲染视图，可以看到PCB的尺寸只有38×28mm，这个尺寸已经接近一颗U盘的大小了，集成度非常高。在如此狭小的板面上面，本人把STM32F103C8T6（LQFP48封装那颗黑色IC）、RT9013 LDO（左上角的SOT-23小芯片）、有源蜂鸣器（左下角圆柱形元件）、三个独立按键（底部三个白色按键体）、RGB灯（右下角白色方形元件）、各种电阻电容（板上分布的0603封装贴片器件）以及对外接口（右侧的5V/GND/TX/RX/VO-/VO+排针）全都摆下了，可以说每一寸面积都被充分利用了，啊，没有半点浪费。

PCB上面本人还印了“庞绿炜 2200810724”的字样，这是为了标识作者信息和学号，便于后期识别和归档。从板子的整体外观来看，颜色均匀、走线整齐、丝印清晰，已经达到了商业级产品的工艺水准。这种小型化、高集成度的PCB设计，决定了整机外壳可以做得非常紧凑便携，符合手持测温仪的产品定位。如果想把这个产品推向市场，38×28mm的核心板可以无缝集成到各种外形的塑料外壳里面，灵活性非常强，也很有商业价值。

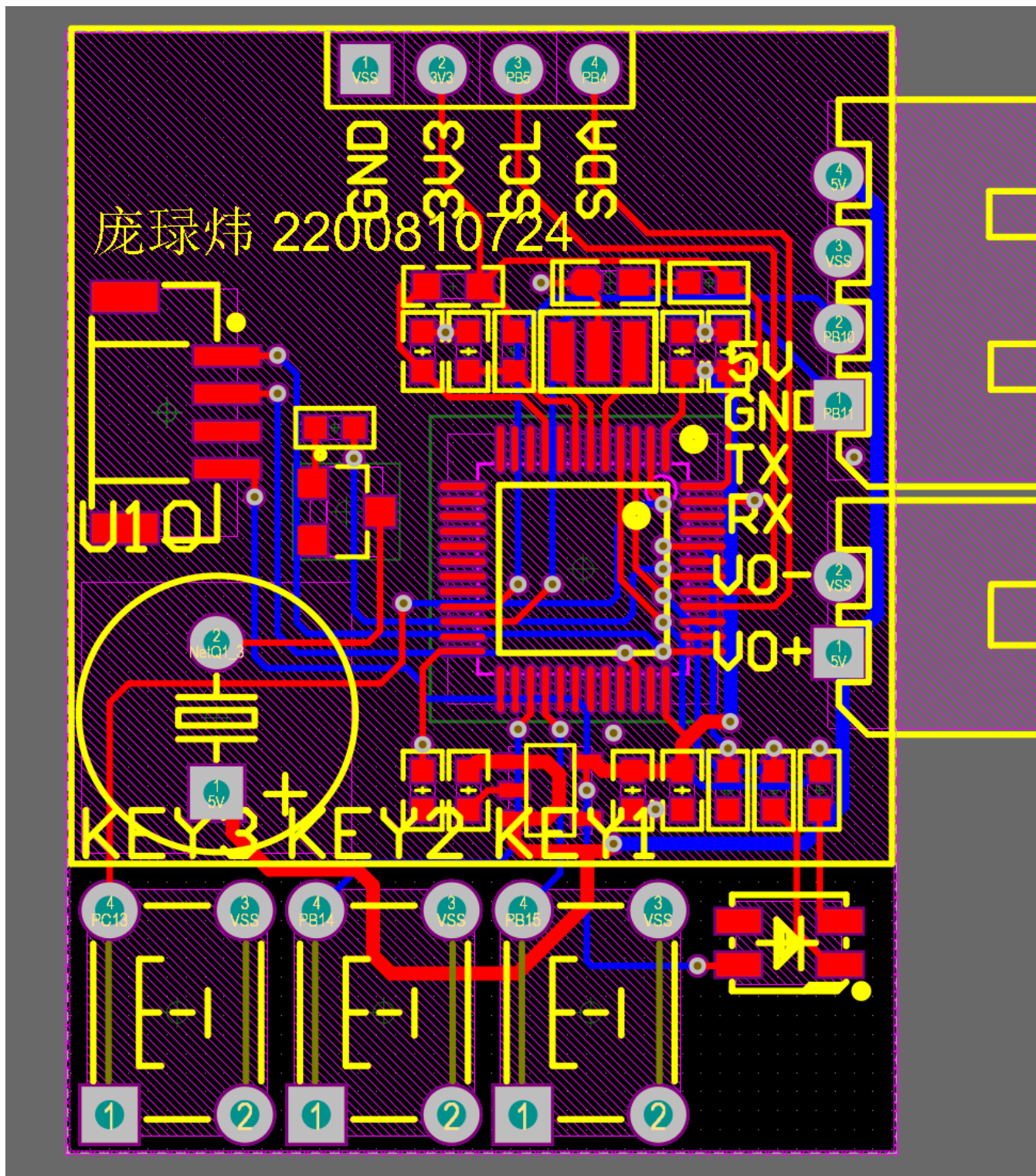


图3-8展示的是PCB的走线视图，可以看到本设计采用的是双层板。其中红色走线是顶层（TOP），蓝色走线是底层（BOTTOM），黄色的线框是丝印层。从走线密度来看，整张板已经接近双层板的极限了，再要加功能基本上就要升级到四层板了。

走线设计方面，本人遵循了几条工程经验。第一，模拟信号（ADC采样线、晶振线）尽量走直线，避免和数字高速信号交叉。第二，电源走线相对粗一些（一般至少20mil），以降低压降和减小寄生电感。第三，关键信号尽量走在同一层，避免频繁切层产生的过孔不连续。第四，整张板的底层尽量铺地，形成完整的GND参考平面，提升抗干扰能力。这些原则在图3-8中都可以观察到对应的体现。

至此，第三章硬件设计就讲解完毕了。本章按照功能模块逐一介绍了STM32最小系统、LDO稳压电源、有源蜂鸣器报警、RGB灯指示、三按键输入、电池电压采集这六个核心电路的设计要点，并展示了最终的PCB成果。整体来看，本设计的硬件架构清晰，模块划分合理，PCB集成度高，为后续的软件开发奠定了坚实的物质基础。下面就让我们进入第四章，看看运行在这块小小PCB上面的软件系统是如何把硬件的各项能力组织起来，最终呈现给用户一个完整的红外测温产品的吧。

第四章 软件设计

经过第三章的硬件设计之后，本人手里已经有了一块功能完整的红外测温仪PCB板。但是裸板是不能工作的，啊，硬件只是骨架软件才是灵魂。要让这块板子真正成为一个能够测温、能够显示、能够报警、能够节能的智能设备，还需要为它编写一套与之匹配的软件系统。本章就来详细介绍本设计的软件实现，重点在于把“代码是怎么思考的”讲清楚，而不是简单地把代码贴出来再翻译一遍注释。

4.1 软件总体架构设计

在动手写第一行代码之前，本人花了相当长的时间去思考软件应该怎么组织。一个嵌入式软件项目，如果从一开始没有想好分层结构和命名规范，越往后写就越乱，最后不可避免地变成一锅粥，自己都看不懂自己写的代码。所以本人参考了若干工业软件的最佳实践，结合本项目的实际需求，设计了一套分层式的软件结构。

4.1.1 FreeRTOS操作系统

本设计选用了FreeRTOS作为软件的运行时操作系统。FreeRTOS是一款轻量级的实时操作系统，开源免费、生态成熟、占用资源少，非常适合STM32F103C8T6这种小型MCU的应用。本人没有像很多入门项目那样去裸跑一个大while循环，而是引入FreeRTOS的根本原因，是想把不同周期的任务隔离开，让它们各自独立运行而不互相干扰。

具体来说，本设计中创建了两个FreeRTOS任务。第一个任务叫task_system，调度周期是10ms，主要负责系统级别的周期性维护工作，比如更新基础库内部的状态机、采样按键、刷新通信链路等等。第二个任务叫task_user1，调度周期是2ms，承担本项目主要的业务逻辑，包括温度采集、显示刷新、报警判断、休眠管理这一系列功能。两个任务通过FreeRTOS的osDelay()函数进行节拍同步，操作系统会自动在它们之间进行时间片的切换，无需本人手工管理。

为什么会把基础周期定为2ms这么短呢，啊，这个周期是经过本人深思熟虑的。如果周期太长比如100ms，按键检测的响应速度会很慢，用户按键后要等100ms才能看到反馈，体验差。如果周期太短比如100us，OLED刷屏频繁会导致功耗暴增。2ms这个值是一个综合考虑下来的折中，既保证了按键检测的快速响应，又不会给系统带来过大的负担。同时2ms的整数倍可以方便地拼出10ms（5×2ms）、100ms（50×2ms）、500ms（250×2ms）这些常用的周期，覆盖各种不同周期任务的需求。

4.1.2 STM32基础外设配置

在进入具体功能软件设计之前，本人先把STM32的几个基础外设的配置情况交代一下。整个外设的配置过程是通过STM32CubeMX图形化工具完成的，CubeMX会根据本人勾选的外设和参数自动生成HAL库的初始化代码，省去了手工查阅寄存器手册的繁琐过程。

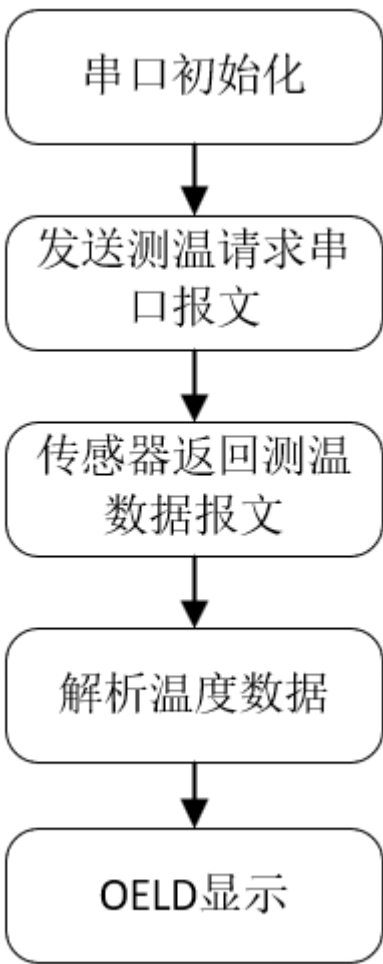
本设计用到的外设主要有以下几类。首先是系统时钟，配置外部8MHz晶振经过PLL九倍频得到72MHz主时钟SYSCLK，APB1时钟36MHz，APB2时钟72MHz。其次是GPIO，把PA、PB、PC的若干引脚分别配置为输入（按键）、输出（RGB灯）、复用（USART/I2C）这几种模式。再然后是USART3，作为和红外测温模块通信的串口，波特率9600，8N1格式，开启RX中断。然后是I2C1，作为驱动OLED的接口，时钟频率400kHz。然后是ADC1，单通道单次转换模式，采样时间55.5个时钟周期，用于采集电池分压电压。最后是TIM4，配置为通用定时器，PWM输出模式，预留给蜂鸣器使用。所有这些外设的初始化代码都在main函数的MX_xxx_Init()调用序列中依次执行，本人不需要操心底层细节。

4.3 红外测温传感器驱动设计

红外测温传感器的驱动是本项目最核心的驱动模块之一，啊，毕竟这是一台测温仪，温度数据从哪里来全靠这个驱动。本人把这个驱动单独写在driver_GY615.c这个文件中，对外只暴露三个API接口，分别是DRIVER_GY615_Init()、DRIVER_GY615_Request()、DRIVER_GY615_GetInfo()。下面让笔者结合代码详细讲解这个驱动的实现思路。

4.3.1 驱动通信流程概述

GY-614V3传感器和STM32之间通过UART串口进行通信，波特率默认9600。通信的基本流程如图4-1所示。



从图中可以看到，整个通信流程分为五步，分别是串口初始化、发送测温请求帧、传感器返回测温数据帧、解析温度数据、最后送到OLED显示。每一步本人都用对应的函数进行封装，整个流程清晰直观。

4.3.2 通信协议帧格式定义

GY-614V3模块采用的是类Modbus协议，本人通过查阅模块手册得知其读寄存器响应帧的格式如下。第一个字节是器件地址（DevAddr，本设计中固定为0x12），第二个字节是功能码（FuncCode，读寄存器=0x03），第三个字节是起始寄存器号（StartReg，本设计中读温度数据从0x07开始），第四个字节是数据字节数（Count，本设计中读7个寄存器），紧接着是Count个数据字节（Data×Count），最后一个字节是校验和（Checksum，前面所有字节累加和的低8位）。

具体到温度数据布局，从0x07寄存器开始连续读7个字节的话，依次得到的内容是发射率E (Buf[0])、目标温度高字节TO_H (Buf[1])、目标温度低字节TO_L (Buf[2])、环境温度高字节TA_H (Buf[3])、环境温度低字节TA_L (Buf[4])、人体温度高字节BO_H (Buf[5])、人体温度低字节BO_L (Buf[6])。温度数值的物理意义是把高低字节拼成一个16位无符号数后，再除以100，单位是摄氏度。比如收到的TO_H=0x0E、TO_L=0x65，拼成0x0E65=3685，除以100就是36.85摄氏度。

4.3.3 解析状态机的设计

由于UART是逐字节到达的，而且不知道什么时候开始什么时候结束，本人为这个传感器设计了一个有限状态机来解析收到的字节流。状态机一共有六个状态，分别是等待器件地址 (GY615_PARSE_WAIT_ADDR)、等待功能码 (GY615_PARSE_WAIT_FUNC)、等待起始寄存器 (GY615_PARSE_WAIT_REG)、等待数据字节数 (GY615_PARSE_WAIT_CNT)、接收数据字节 (GY615_PARSE_RECV_DATA)、等待校验和 (GY615_PARSE_WAIT_CSUM)。状态转移的规则严格按照协议帧格式定义，每收到一个字节就检查当前状态，决定下一步走向哪里。

下面是状态机的核心实现代码。

```
static void __DRIVER_GY615_UartByteCallback(uint8_t port, uint8_t byte){
    if(port != GY615_DEP_UART_PORT) return;

    switch(gy615Ctx.parseState){
        case GY615_PARSE_WAIT_ADDR:
            if(byte == GY615_DEV_ADDR){
                gy615Ctx.parseChecksum = (uint16_t)byte;
                gy615Ctx.parseState = GY615_PARSE_WAIT_FUNC;
            }
            break;

        case GY615_PARSE_WAIT_FUNC:
            if(byte == GY615_FUNC_READ){
                gy615Ctx.parseChecksum += (uint16_t)byte;
                gy615Ctx.parseState = GY615_PARSE_WAIT_REG;
            } else {
                __DRIVER_GY615_ResetParser();
            }
            break;
        /* 其余状态省略 */
    }
}
```

这个回调函数是被串口驱动以中断方式调用的，每收到一个字节就调用一次，完全不阻塞主程序的运行。状态机内部使用一个全局静态结构体gy615Ctx来保存中间状态，包括当前所处的状态、已接收的数据缓冲区、已接收字节数、累计校验和等等。每帧解析完成（无论成功与否）都会调用__DRIVER_GY615_ResetParser()把状态机复位到初始态，准备接收下一帧。

这种状态机解析的方法比“等够N个字节再一起解析”的方法要鲁棒得多。因为UART通信会偶尔丢字节或者乱序，如果用攒够一帧再处理的方式，一旦某个字节丢了整帧就完全乱套了。而状态机方法每收到一个字节都会立即判断它的合法性，遇到不合法的字节就立即丢弃并复位状态机，下一帧又能够重新开始解析，这种“自我恢复”的能力对于嵌入式通信的可靠性至关重要。

4.3.4 数据解析函数

当状态机走到WAIT_CSUM状态并且校验和验证通过之后，会调用__DRIVER_GY615_ParseData()函数对缓冲区中的字节做最终的物理意义解析。这个函数的代码如下。

```
static void __DRIVER_GY615_ParseData(void){
    uint16_t rawTo;
    uint16_t rawTa;
    uint16_t rawBo;

    if((gy615Ctx.parseReg != GY615_READ_START_REG) ||
        (gy615Ctx.parseCount != GY615_READ_COUNT)){
        return;
    }

    gy615Ctx.infoCache.emissivity = gy615Ctx.parseDataBuf[0];

    rawTo = ((uint16_t)gy615Ctx.parseDataBuf[1] << 8U) | (uint16_t)gy615Ctx.parseDataBuf[2];
    rawTa = ((uint16_t)gy615Ctx.parseDataBuf[3] << 8U) | (uint16_t)gy615Ctx.parseDataBuf[4];
    rawBo = ((uint16_t)gy615Ctx.parseDataBuf[5] << 8U) | (uint16_t)gy615Ctx.parseDataBuf[6];

    gy615Ctx.infoCache.targetTempC = (float)rawTo / 100.0f;
    gy615Ctx.infoCache.ambientTempC = (float)rawTa / 100.0f;
    gy615Ctx.infoCache.bodyTempC = (float)rawBo / 100.0f;
    gy615Ctx.infoCache.hasNewData = 1U;
}
```

函数的输入是已经填好的parseDataBuf缓冲区，输出是更新到infoCache结构体里的物理温度值。函数先做了一个保护性检查，如果起始寄存器或者数据字节数不符合预期就直接返回不做处理。然后逐一把高低字节拼接成16位无符号整数（rawTo、rawTa、rawBo），再除以100.0f得到摄氏度温度值。最后把hasNewData标志位置1，告诉调用方"有新数据可以读了"。

这里有一个细节本人想拿出来说一下，那就是为什么除数要写成100.0f而不是100。在C语言里面，100是int类型而100.0f是float类型，如果用100做除数，编译器可能会先把rawTo转成int做整数除法，丢失小数部分，然后再赋值给float变量。而用100.0f做除数，编译器会自动把整个表达式提升为float做浮点除法，保留小数精度。这种类型相关的小细节如果不注意，会导致温度精度白白损失。

4.3.5 请求函数与获取函数

driver_GY615对外暴露的另外两个核心API分别是DRIVER_GY615_Request()和DRIVER_GY615_GetInfo()。前者负责向传感器发送一帧"请读取0x07开始的7个寄存器"的请求，后者负责把最新解析到的温度数据返回给调用方。它们的代码如下。

```
void DRIVER_GY615_Request(void){
    uint8_t frame[5];
    uint16_t checksum;

    frame[0] = GY615_DEV_ADDR;
    frame[1] = GY615_FUNC_READ;
    frame[2] = GY615_READ_START_REG;
    frame[3] = GY615_READ_COUNT;
```

```

        checksum = (uint16_t)frame[0] + (uint16_t)frame[1] +
                    (uint16_t)frame[2] + (uint16_t)frame[3];
        frame[4] = (uint8_t)(checksum & 0xFFU);

        GY615_DEP_UART_SEND_RAW(frame, (uint16_t)sizeof(frame));
    }

    uint8_t DRIVER_GY615_GetInfo(GY615Info_t *info){
        if(info == NULL) return 0U;
        if(gy615Ctx.infoCache.hasNewData == 0U) return 0U;

        *info = gy615Ctx.infoCache;
        gy615Ctx.infoCache.hasNewData = 0U;
        return 1U;
    }

```

Request函数构造了一个5字节的请求帧（地址+功能码+起始寄存器+数据数+校验和），然后通过串口发送出去。GetInfo函数则是先检查传入指针是否有效，然后看看缓存里有没有新数据，有就拷贝出来并清除新数据标志位，没有就返回0表示无效。这种"请求-获取"分离的设计模式，让上层应用可以采用流水线方式工作，发出请求之后不必死等响应，可以先去做别的事情，过一段时间再回来取结果，效率更高。

4.3.6 初始化函数

driver_GY615的初始化函数比较简单，主要做两件事，一个是清零内部上下文，另一个是把字节回调函数注册到串口驱动上面。代码如下。

```

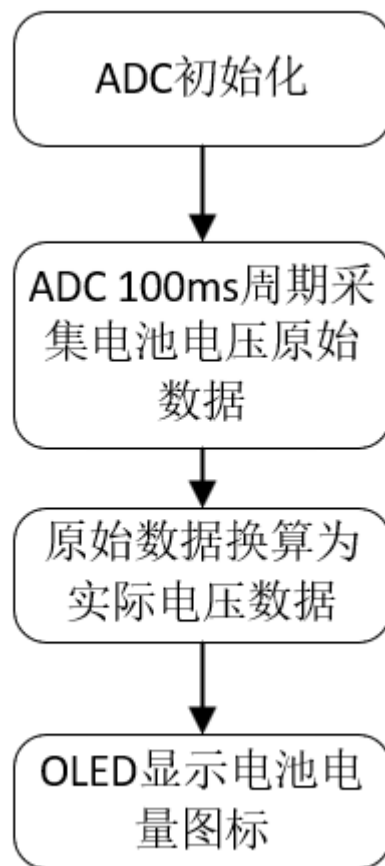
void DRIVER_GY615_Init(void){
    memset(&gy615Ctx, 0, sizeof(gy615Ctx));
    __DRIVER_GY615_ResetParser();
    GY615_DEP_UART_SET_CUSTOM_CB(__DRIVER_GY615_UartByteCallback);
}

```

初始化之后，每次串口收到一个字节都会自动调用__DRIVER_GY615_UartByteCallback，触发状态机解析，整个数据接收流程就完全自动化了。

4.4 电池电压采集功能软件设计

电池电量显示是用户体验中很重要的一环，啊，没人愿意在不知道电量的情况下使用一台手持设备。本设计通过ADC采集电池分压电压，再经过软件换算和滤波，最终得到一个0~100%的电量百分比显示在OLED上面。整个流程如图4-2所示。



从图中可以看到，流程包括ADC初始化、100ms周期采集电池电压原始数据、原始数据换算为实际电压、最后OLED显示电量图标这几个步骤。下面分别讲解这几个步骤的软件实现。

4.4.1 ADC驱动的初始化

ADC的硬件配置由CubeMX自动生成，本人只需要在driver_adc.c里面写一个DRIVER_ADC_Init函数，调用HAL库启动ADC即可。ADC采用的是单通道单次转换模式，每次软件触发一次，返回一个12位的原始数字值，对应0到4095的整数。

4.4.2 周期采样与原始数据换算

ADC的采样工作是由system任务以10ms周期触发的，每次采样到的原始值会被存到adcDriverInfo结构体里面。在用户任务里面，每500ms调用一次DRIVER_ADC_UpdateBattPercent()，把原始ADC值换算为电池电压百分比。换算公式如下。

设ADC原始读数为adcRaw（0~4095），分压比为 $k=R_{11}/(R_{10}+R_{11})\approx 0.357$ ，参考电压Vref=3.3V，那么电池实际电压V_BAT等于 $adcRaw \times Vref / (4096 \times k)$ 。再根据满电8.4V对应100%、截止6.0V对应0%这个线性映射关系，就可以得到电量百分比。

4.4.3 电量图标的显示逻辑

电量百分比拿到之后，user1TaskInfo.battPercent就会被更新。在OLED刷新函数User1Task_RefreshOled()里面，通过调用User1Task_GetBattBar()函数，把百分比映射到五种不同的图标字符串，分别对应0~9%、10~24%、25~49%、50~74%、75~100%这五个档位。这种离散显示比连续显示要直观得多，用户一眼就知道电量大概处于哪个范围。

另外当电量低于10%时，本人特意做了一个闪烁效果。具体做法是利用taskCnt每500ms翻转一次显示，让电量图标在"显示"和"全空格"之间交替，从视觉上给用户一个强烈的低电量警示。代码片段如下。

```
battBlink = (uint8_t)((user1TaskInfo.taskCnt / 250U) % 2U);
if((pct < 10U) && (battBlink == 0U)) {
    barStr = "      "; /* 低电量闪烁：熄灭图标 */
} else {
    barStr = __User1Task_GetBattBar(pct);
}
```

这种小细节看起来不起眼，但是对用户体验的提升相当明显。本人觉得做产品就是要在这种小地方下功夫。

4.5 用户主任务设计

讲完了底层驱动，下面就来介绍贯穿全局的应用层主任务task_user1。这个任务是整个软件系统的"指挥中心"，所有的业务逻辑都在它里面运行。

4.5.1 任务初始化函数

任务启动时第一个被调用的是user1TaskInit()，它负责把所有依赖的驱动模块初始化好，并把应用层的状态变量设置成合理的初值。代码如下。

```
void user1TaskInit(void) {
    DRIVER_GY615_Init();
    osDelay(100U);
    DRIVER_OLED_Init();
    DRIVER_ADC_Init();

    user1TaskInfo.taskCnt          = 0U;
    user1TaskInfo.tempLatest       = 0.0f;
    user1TaskInfo.tempDisplay      = 0.0f;
    user1TaskInfo.bodyTempLatest   = 0.0f;
    user1TaskInfo.bodyTempDisplay  = 0.0f;
    user1TaskInfo.alarmThreshHigh10 = 375; /* 初始上限37.5°C */
    /* ... 其余字段省略 */

    /* 上电首次温度采集 */
    {
        GY615Info_t snapshot;
        DRIVER_GY615_Request();
        osDelay(50U);
        if(DRIVER_GY615_GetInfo(&snapshot) != 0U) {
            /* 拷贝有效温度作为初始显示值 */
        }
        DRIVER_GY615_Request();
    }

    __User1Task_Refresholed();
}
```

这里有几个细节本人想说明一下。第一个是OLED初始化前的osDelay(100U)，这是因为SSD1306数据手册要求VCC稳定后大约100ms才能开始接受I2C命令，如果不加这个延时，在冷启动的时候OLED可能会因为电源还没完全稳定就收到初始化命令而无视，导致屏幕黑屏。本人之前就栽过这个跟头，加了延时之后就再也没出过问题。

第二个细节是上电首次温度采集那一段，本人特意发起了一次温度请求然后等待50ms再读，目的是让开机瞬间OLED上面就显示一个真实的温度值，而不是停留在初始的0.0°C上面。如果不做这一步，用户开机后会看到屏幕显示0.0°C约100ms之后才跳到正常温度，体验比较突兀。这种"开机即可用"的设计，对产品好感度的提升很大。

第三个细节是alarmThreshHighX10被初始化为375，对应37.5°C。这个值是参考人体正常体温上限设置的，符合任务书中"32°C到45°C测温范围内精度0.2°C"的需求语境。当然用户可以通过KEY2和KEY3按键随时调整这个阈值。

4.5.4 按键事件处理逻辑

三个按键各自有不同的功能。KEY1是"测温确认"键，按下后会把后台采集到的最新温度tempLatest同步到显示用的tempDisplay，同时重置显示保持计时器和无操作计时器，调用__User1Task_EvalAlarm()重新评估报警状态，最后刷新OLED。KEY2是"阈值上调"键，每按一次把alarmThreshHighX10加1（也就是+0.1°C），上限60°C。KEY3是"阈值下调"键，逻辑类似KEY2但方向相反，下限0°C。

这里有一个小心思笔者想分享一下。本人特意把"测温确认"和"实时采集"分开来设计，也就是后台一直在以100ms周期采集tempLatest，但是显示的tempDisplay只在用户按下KEY1的时候才更新一次。这样做的目的是模拟商用红外测温仪的"按键测温"用户体验，用户对准被测物体按一下KEY1，屏幕上就显示一个固定的测温结果，不会因为传感器轻微晃动而数字一直跳变，给用户一种"测得准"的心理感受。

报警判断也只在按下KEY1或者修改阈值的时候触发，而不是在后台100ms周期里面持续判断，这是为了避免传感器对准空气时（温度可能很低或者很怪异）误触发报警。这种"用户主动确认才生效"的设计哲学，贯穿了本人整个软件实现。

4.5.5 报警评估函数

__User1Task_EvalAlarm()是负责判断当前显示温度是否超出报警阈值的函数，代码如下。

```
static void __User1Task_EvalAlarm(void) {
    float highThresh;

    if(user1TaskInfo.isValidTemp == 0U) {
        return;
    }

    highThresh = (float)user1TaskInfo.alarmThreshHighX10 / 10.0f;

    if(user1TaskInfo.tempDisplay > highThresh) {
        user1TaskInfo.alarmActive = 1U;
        boardInfo.buzzTimeMs      = 0xFFFFU;
        DRIVER_BOARD_RgbSet(BOARD_RGB_R, 1U);
        DRIVER_BOARD_RgbSet(BOARD_RGB_G, 0U);
    } else {
        user1TaskInfo.alarmActive = 0U;
        boardInfo.buzzTimeMs      = 0U;
        DRIVER_BOARD_RgbSet(BOARD_RGB_R, 0U);
        DRIVER_BOARD_RgbSet(BOARD_RGB_G, 1U);
    }
}
```

函数先做一个保护，如果还没有有效温度直接返回不动作。然后把整数表示的阈值（×10存储是为了避免浮点比较误差）转换回浮点的摄氏温度值。再用tempDisplay和高阈值比较，超过就报警，否则正常。报警时让蜂鸣器持续鸣叫（buzzTimeMs=0xFFFFU即65535ms最大值，相当于无限响）、红灯亮、绿灯灭。正常时关蜂鸣器、红灯灭、绿灯亮。

这里要解释一下为什么alarmThreshHighX10要存成“×10的整数”而不是直接存成浮点。因为按键每按一次只调整0.1℃，如果用float存储会有累加误差（连续加100次0.1f并不严格等于10.0f），用整数存储就完全没有这个问题。这种“整数化定点表示”的小技巧在嵌入式编程中非常常用，本人在很多地方都用了类似的手法。

4.6 低功耗管理驱动设计

低功耗是手持设备的核心特性之一，啊，本设计的低功耗管理由driver_power.c这个驱动统一处理。该驱动的代码非常简洁，但是背后的原理值得说道。

```
void DRIVER_POWER_EnterSleep(void) {
    HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);
}

void DRIVER_POWER_WakeUp(void) {
    /* 预留：如需唤醒后执行外设重初始化，在此添加 */
}
```

DRIVER_POWER_EnterSleep()函数调用了HAL库提供的HAL_PWR_EnterSLEEPMode接口，参数指定主调压器保持开启、通过WFI（Wait For Interrupt）指令进入睡眠。STM32F103进入SLEEP模式后CPU时钟停止，内核停止取指执行，但是外设时钟和RAM内容都保持着，整机功耗从正常运行的30mA左右降到几个mA的量级。

这里特别要说明的是，由于本人是在FreeRTOS任务里面调用WFI，每1ms就会有一个SysTick中断进来唤醒CPU执行调度，CPU又会把任务切回来再进WFI再被唤醒。这个看起来有点奇怪的“反复进出睡眠”模式，其实是FreeRTOS低功耗设计的一种基本套路，叫做Tickless之前的过渡方案。它能够让CPU在没有用户操作的时候大部分时间都处于低功耗状态，又能够及时响应按键中断和系统时钟。

4.6.1 唤醒机制

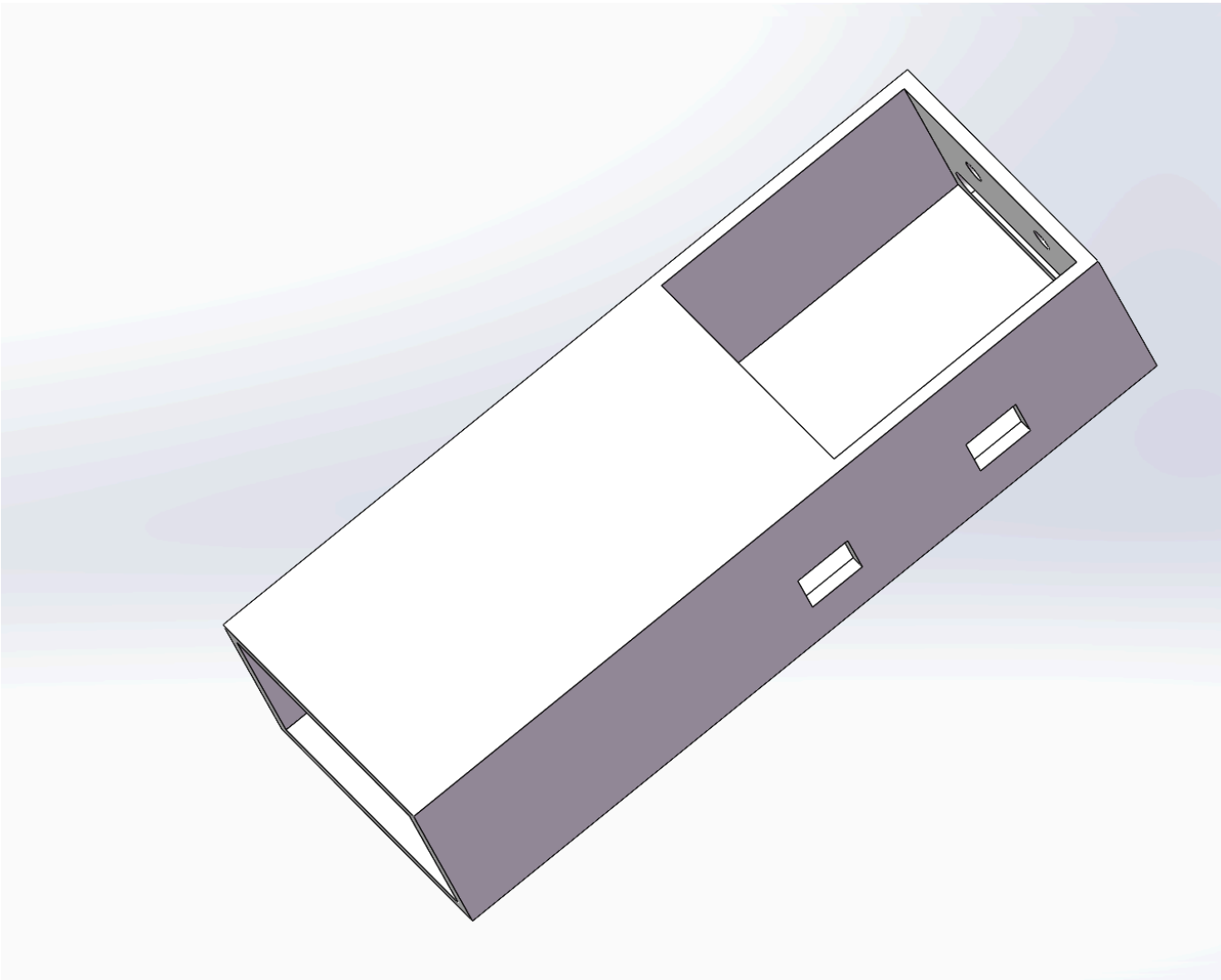
那么休眠之后怎么醒过来呢，主要靠两条路径。第一是按键中断，本人把三个按键都配置成了EXTI外部中断，按下按键会立即产生中断把CPU唤醒。第二是SysTick中断，FreeRTOS的1ms节拍中断会持续地唤醒CPU执行任务调度。前者用于响应用户主动操作，后者保证操作系统不会完全停摆。

唤醒之后的恢复工作由_User1Task_HandleWakeUp()函数处理，它会调用DRIVER_POWER_WakeUp()做一些必要的外设恢复（当前版本预留为空，因为SLEEP模式不会丢失外设状态），然后清sleepFlag、清idleCnt、亮起绿灯、开OLED显示，最后刷新OLED。从用户视角看就是按一下任意按键，屏幕马上亮起来恢复到正常工作状态，体验非常顺滑。

第五章 机械外观设计

软硬件设计完成之后，最后一道工序就是把PCB板和电池装进一个合适的外壳里面，让产品具备真正的“商品形态”。本设计的机械外壳采用SOLIDWORKS 2025进行三维建模，最终的整体外观如图5-1所示。

5.1 整机外壳设计



从图中可以看到，本设计采用的是一种类似"枪型"的人体工学造型，整体呈现长方体加倾斜过渡的形态。外壳由ABS塑料一体化成型，内部留有专门的腔体用于安放各个核心组件。

啊，下面让笔者来详细介绍各个开口和腔体的功能划分。从图5-1可以观察到，外壳的顶部有一个大的矩形开口，这个开口是专门为PCB板设计的安装位置。本人在前面第三章已经介绍过，整张PCB的尺寸是38×28mm，外壳顶部的矩形开口尺寸略大于PCB，并且内壁设有四个小柱子用于通过自攻螺丝固定PCB。这种"柱子+螺丝"的安装方式比单纯的卡扣要可靠得多，可以保证PCB在使用过程中不会因为震动而松动。

OLED屏幕是通过排针插接到PCB上面的，这种可插拔的设计使得当OLED不慎损坏的时候可以方便地更换，不需要拆焊PCB。本人在外壳上面OLED对应的位置开了一个矩形显示窗，窗口的尺寸刚好和0.96寸OLED的可视区相匹配，使得OLED屏幕嵌入外壳后正好和外壳整体表面平直，没有突出也没有凹陷，外观非常协调。这种"齐平式"安装的好处不只是好看，更重要的是不会出现凸起的边缘容易磕碰损坏的问题。

外壳的底部有一个开口，这个开口是用来放置7.4V锂电池的电池仓。本设计采用的是两节18650锂电池串联组成7.4V电池组的方案，电池仓的尺寸正好可以容纳两节18650电池，并且配有电池触片用于电池正负极的连接。底部开口设计成可以滑动开合的电池盖，用户在更换电池时只需要把电池盖向后推开就可以露出电池仓，操作非常方便。

红外测温传感器GY-614V3被安装在整机的最顶部位置，这是经过本人深思熟虑的设计决定。把传感器放在最顶端的好处主要有以下几个方面。第一，红外测温的工作原理是接收被测物体发射的红外辐射，传感器和被测物体之间不能有任何遮挡物，把传感器放在最顶端可以最大限度地避免被外壳本体遮挡的可能。第二，从用户使用习惯来说，握持本设备时一般是手柄在下、传感器在上指向被测物体，这种"枪指目标"的姿势符合直觉，新用户拿到产品后基本上不需

要看说明书就知道怎么用。第三，传感器在顶部位置时，距离被测物体的距离更近一些，有利于提高测温精度。

外壳左侧还可以看到一些方形的开口，这些是按键孔，对应着PCB上面的三个独立按键。按键孔的尺寸略大于按键键帽，使得按键能够正常按压而不会卡顿。本人在按键孔的内壁还做了一些导向斜面，避免按键在按下时和孔壁产生侧向摩擦影响手感。

从制造工艺上面来看，整个外壳采用ABS塑料注塑成型。ABS塑料的优点是强度高、加工性好、表面易于上色，是消费电子产品最常用的外壳材料。如果要批量生产的话，开一套注塑模具的成本大约在几万元人民币的量级，分摊到每个产品上面成本可以控制在几元钱以内，非常适合商业化推广。

外壳的整体尺寸大约是120mm（长）×40mm（宽）×30mm（厚），体积小巧，重量轻盈，单手握持非常舒适。这个尺寸已经接近市面上同类型商业红外测温仪的水平，可以说本设计的机械结构在便携性方面达到了商品级标准。